

Object-Oriented Programming with C++

Complete Course Module | Chapters 1–11 | 41 Credit Hours

Course Objectives

- Understand core principles of Object-Oriented Programming
- Apply OOP concepts using C++ syntax and features
- Design and implement classes, objects, inheritance hierarchies, and polymorphic behavior
- Use advanced features: templates, exception handling, file I/O
- Solve Purwanchal University exam-level problems in C++

Chapter Overview

Ch.	Title	Hrs
1	Introduction to OOP	3
2	Basic Syntax of C++	2
3	Functions in C++	3
4	Classes and Objects	5
5	Friend Function	4
6	Constructor & Destructor	4
7	Operator Overloading	5
8	Inheritance	5
9	Polymorphism	4
10	File Handling	5
11	Advanced Topics	5

Introduction to OOP

- OOP vs Procedural-Oriented Programming (POP)
- Core Features: Objects, Classes, Encapsulation, Abstraction
- Inheritance, Polymorphism, Dynamic Binding, Message Passing
- Applications and Benefits of OOP

OOP vs POP

Procedural (POP)

- Divided into functions
- Global data accessible anywhere
- Top-down design
- Less secure — open data
- Hard to maintain at scale
- Examples: C, Pascal

Object-Oriented (OOP)

- Divided into objects
- Data encapsulated in objects
- Bottom-up design
- Secure — data hidden
- Easy to maintain and extend
- Examples: C++, Java, Python

The 8 Pillars of OOP

- Objects — Real-world entities with state and behavior
- Classes — Blueprints that define objects
- Abstraction — Show essentials, hide complexity
- Encapsulation — Bundle data + methods, control access
- Inheritance — Derive new classes from existing ones
- Polymorphism — Same interface, different behavior
- Dynamic Binding — Function resolved at runtime
- Message Passing — Objects communicate via method calls

Ch.1 Analogy & PU Question



Real-World Analogy

Think of a Car Dealership: The 'Car' class is the blueprint. Each specific car sold is an object. The engine mechanism is hidden (encapsulation). A Tesla inherits from Car but adds electric-specific behavior (inheritance). Pressing the accelerator works on any car regardless of fuel type (polymorphism).



Probable PU Exam Question

Q: What is OOP? Explain any four features with examples. Differentiate OOP from POP. [PU Exam 2022]

Basic Syntax of C++

- C++ Program Structure
- new / delete operators for dynamic memory
- const, Enumeration, Reference Variables
- Scope Resolution Operator (::)
- Manipulators: setw, setprecision, setfill, endl
- Type Conversion: Implicit and Explicit

Dynamic Memory: new & delete

```
#include <iostream>
using namespace std;

int main() {
    int *p = new int;      // Heap allocation
    *p = 42;
    cout << *p;
    delete p;              // Free memory

    int *arr = new int[5]; // Array allocation
    arr[0] = 10;
    delete[] arr;          // Free array
    return 0;
}
```

💡 Always delete what you new Memory leaks crash programs

Reference Variables & Scope Resolution

```
int val = 100;    // Global variable

void show() {
    int val = 50;    // Local variable
    cout << val;      // Prints 50 (local)
    cout << ::val;    // Prints 100 (global)
}

// Reference variable usage
int x = 10;
int &ref = x;      // ref is alias for x
ref = 20;          // x is now 20
cout << x;         // Output: 20
```

💡 Reference must be initialized at declaration and cannot be re-seated

Ch.2 Analogy & PU Question



Real-World Analogy

Reference variables are like nicknames: 'Robert' and 'Bob' refer to the same person. Changing one changes the other. The scope resolution operator (::) is like a city code — you use 'Kathmandu::Ram' to distinguish from 'Pokhara::Ram'.



Probable PU Exam Question

Q: Explain reference variables and scope resolution operator with examples. Write a program using setw, setprecision, and setfill. [PU Exam 2021]

IN

Functions in C++

- Inline Functions — eliminate call overhead
- Function Overloading — same name, different signatures
- Default Arguments — trailing parameters with default values

Functions

A function is a group of statements that together perform a task. Every C++ program has at least one function, `main()`, which is the starting point of the program. Functions can be defined by the user or provided by libraries.

Why Use Functions?

Modularity: Break down complex problems into smaller, manageable pieces.

Reusability: Write code once and use it multiple times, even in different programs.

Organization: Make the code more organized and easier to read and maintain.

Basic Syntax

A function in C++ is defined using the following syntax:

```
// Basic Syntax of Function
returnType functionName(parameterList) {
// body of the function
}
```

- **returnType:** The data type of the value the function returns. If the function does not return a value, this is specified as `void`.
- **functionName:** The name of the function.
- **parameterList:** A list of parameters (arguments) that the function takes, enclosed in parentheses. This is optional; a function may have no parameters.

Functions

Types of Functions

1. Standard Functions

Standard functions are those provided by C++ libraries, such as `std::cout` for output and `std::cin` for input.

2. User-Defined Functions

User-defined functions are written by the programmer to perform specific tasks. The example above illustrates a simple user-defined function.

```
#include <iostream>
using namespace std;

// Function declaration
int add(int, int);

int main() {
    int result = add(5, 3); // Function
    call
    cout << "The sum is " << result <<
    endl;
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Functions

Function Components

Return Type

A function may return a value. The return type is the type of value the function sends back to the caller. If a function does not return a value, its return type is void.

Parameters

Parameters (or arguments) are the values passed to the function. They are optional; a function can have none. Parameters allow you to pass data into a function.

- Pass-by-value
- Pass-by-reference

Function Body

The function body contains the statements that define what the function does. It's enclosed in curly braces {}.

Functions

Default Arguments

Default arguments in C++ functions are a feature that allows us to specify default values for parameters. These defaults are used when a corresponding argument is omitted from a function call. This feature enhances function flexibility, simplifies function calls, and maintains code clarity by minimizing the need for overloaded functions for different cases.

```
returnType functionName(parameterType1  
parameterName1 = defaultValue1, ...);
```

Understanding Default Arguments

If the function is called without an argument for a parameter with a default value, the default value is automatically used by the function. Parameters with default values must be the rightmost parameters in the function declaration.

```
#include <iostream>
using namespace std;

// defining the default arguments
void display(char c = '*', int count = 3) {
    for(int i = 1; i <= count; ++i) {
        cout << c;
    }
    cout << endl;
}

int main() {
    int count = 5;

    cout << "No argument passed: ";
    display();

    cout << "First argument passed: ";
    display('#');

    cout << "Both argument passed: ";
    display('$', count);

    return 0;
}
```

Functions Overloading

Multiple functions with same name but different parameters.

Function overloading is a feature in C++ that allows multiple functions to have the same name but with different parameters (either in type, number, or both). This capability enhances the expressiveness and readability of the programming language, allowing functions to be used in various contexts with different types of input.

Functions Overloading

Explanation of Program:

```
double area(double radius) {
```

This version of the area function calculates the area of a circle. It takes one double parameter representing the radius of the circle and returns its area as a double.

```
double area(double length, double width) {
```

This version calculates the area of a rectangle. It is overloaded to take two double parameters representing the length and width of the rectangle, respectively, and returns the calculated area.

```
cout << "Area of circle (r=5): " << area(5.0) << endl;
```

Calls the first version of the area function with a single argument to calculate the area of a circle.

```
cout << "Area of rectangle (4x5): " << area(4.0, 5.0) << endl;
```

Calls the second version of the area function with two arguments to calculate the area of a rectangle.

```
#include <iostream>
using namespace std;

// Overloaded functions to calculate the area
double area(double radius) { // For circle
return 3.14159 * radius * radius;
}

double area(double length, double width) {
// For rectangle
return length * width;
}

int main() {
cout << "Area of circle (r=5): " <<
area(5.0) << endl;
cout << "Area of rectangle (4x5): " <<
area(4.0, 5.0) << endl;
return 0;
}
```

Functions Overloading

Use Cases for Function Overloading

Handling Different Data Types: Overloaded functions can perform analogous operations on different types of data. For example, you might have a print function that accepts both int and float types, displaying numbers in an appropriate format.

Different Implementations Based on Parameters: You can overload functions to provide different implementations based on the number or types of arguments passed. This is useful in mathematical operations, where the formula might differ based on input dimensions or types.

Improving Code Readability: Using the same function name for similar operations on different data types makes the code more intuitive and easier to understand.

Advantages

Cleaner Code: Function overloading leads to cleaner code by reducing the number of distinct function names for similar operations, making the code easier to read and maintain.

Intuitive Usage: Overloading allows functions to be called in a way that feels natural and intuitive, closely mimicking the versatility seen in human languages.

Compile-Time Polymorphism: It provides a form of polymorphism where the appropriate function is selected at compile time based on the argument types, improving performance by avoiding runtime decision-making.

Inline Function

Inline functions are a powerful feature in C++ designed to optimize the execution speed of programs, particularly for small, frequently called functions. By marking a function as inline, we give a hint to the compiler that it should attempt to embed the function's body directly into each place the function is called, instead of performing a traditional function call. This section explores what inline functions are, their origin, use cases, and the potential risks associated with their use.

Example of Inline Function

```
inline int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

This simple function returns the maximum of two integers. By declaring it as inline, we suggest to the compiler that replacing calls to max with the function's body could be more efficient than a traditional function call, especially if max is used frequently in performance-critical parts of the code.

Inline Function

```
#include <iostream>
using namespace std;

inline int square(int x) {
    return x * x;
}

int main() {
    cout << square(5) << endl; // expands to 5*5
    cout << square(6 + 1) << endl; // expands to (6+1)*(6+1)
    return 0;
}
```

```
#include <iostream>
using namespace std;

inline int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    cout << max(10, 20) << endl;
    cout << max(100, 50) << endl;
    return 0;
}
```

Inline Functions vs Function Overloading

- Inline: Compiler replaces call with function body
- Use for small, frequently called functions
- Avoids function call overhead (stack frame, jump, return)
- Hint only — compiler may ignore for complex functions
- Overloading: Same function name, different parameter list
- Resolved at COMPILE TIME (early binding)
- Cannot differ only by return type
- Enables intuitive API design

```
// Inline
inline int sq(int x) { return x*x; }

// Overloaded
int add(int a, int b)    { return a+b; }
float add(float a, float b) { return a+b; }
int add(int a,int b,int c){ return a+b+c; }

// Default args
void info(string n, int age=18,
          string city="Kathmandu") {
    cout << n<<"<<age<<"<<city;
}
```

Ch.3 Analogy & PU Question



Real-World Analogy

Function overloading is like a restaurant 'Combo Meal' that means different things at breakfast vs lunch vs dinner. The name is the same but the behavior changes based on context (arguments). Default arguments are like a standing order at your regular cafe — you say 'the usual' and specific defaults apply.



Probable PU Exam Question

Q: What are inline functions? Write a C++ program demonstrating function overloading and default arguments.
[PU Exam 2023]

Lab Work

Scenario 1: Smart Calculator

Problem Statement

Develop a calculator program that can perform addition on different types of inputs.

Requirements

Create overloaded functions named add() for:

- *Two integers*
- *Two floating-point numbers*
- *Three integers*

Create an inline function square() that returns the square of a number.

Display the result of each operation.

Concepts needed to implement

- *Function Overloading*
- *Inline Function*
- *Basic Arithmetic Operations*

Lab Work

Scenario 2: Employee Salary Management

Problem Statement

A company wants a salary calculation system for different categories of employees.

Requirements

Overload a function calculateSalary() for:

- *Full-time employee (basic salary)*
- *Employee with bonus*
- *Employee with bonus and overtime*

Create an inline function taxDeduction() that calculates 10% tax.

Display gross salary and net salary.

Concepts needed to implement

- *Function Overloading with different parameter lists*
- *Inline Functions*
- *Real-world business application*

Lab Work

Scenario 3: Student Result Processing

Problem Statement

Create a result-processing system for students.

Requirements

Overload a function calculateAverage() for:

Two subjects

Three subjects

Five subjects

Create an inline function isPass() that returns true if average ≥ 40 .

Display average marks and pass/fail status.

Concepts needed to implement

- *Function Overloading with different parameter lists*
- *Inline Functions*
- *Real-world business application*

Classes and Objects

- Visibility Modes: public, private, protected
- Data Members and Member Functions
- Static Members — shared across all objects
- Passing and Returning Objects
- The 'this' Pointer

Introduction

- Objects and Classes are key to understand the Object-Oriented Programming Language (OOPL)/Object-Oriented Technique. Object-Oriented Programming Languages are based on the concept of abstraction modeled by classes and objects.
- Classes and objects are the cornerstones of Object-Oriented Programming (OOP) in C++. They enable programmers to create modular, reusable, and maintainable code. In this article, we will dive deep into understanding what classes and objects are, explore their functionalities, and learn how to implement them in C++ effectively.
- Classification is one of the important concepts of Object-Oriented philosophy including identity, abstraction, encapsulation, inheritance, polymorphism, and persistence.

Object-Oriented uses classification to group objects that have attributes and behaviors in common and classify them into bigger entities. The bigger entity is called a “class”.

Thus, a class defines a group of objects with similar attributes, common operations, a common relationship to the other objects in the class, and common semantics.

Class

A class is set of objects that shares a common definition described by data and methods.

According to the Webster's New World dictionary, a class is defined as:

“A number of people or things grouped together because of certain likeness; kind; sort”. In other words, we can say that class is an object template. Every object under that class has the same data format, definition and responds in the same manner to an operation.

A **class** in C++ is a user-defined data type that encapsulates data members (variables) and member functions (methods) within a single unit. Think of a class as a blueprint or template that defines the structure and behavior of objects. Classes are the foundation of OOP and facilitate encapsulation, abstraction, and data hiding.

Class

Analogy of Classes:

Imagine a class of cars. Cars can have different names, models, and brands, yet they all share common properties like having four wheels, a speed limit, and mileage. In this analogy:

- ✓ The **Car** represents the class.
- ✓ The common properties such as wheels, speed limits, and mileage are the **data members**.
- ✓ Actions like accelerating, braking, and honking are the **member functions**.

Defining a Class in C++

A class in C++ is defined using the class keyword followed by the class name. The body of the class contains the data members and member functions, enclosed in curly braces {}.

Class

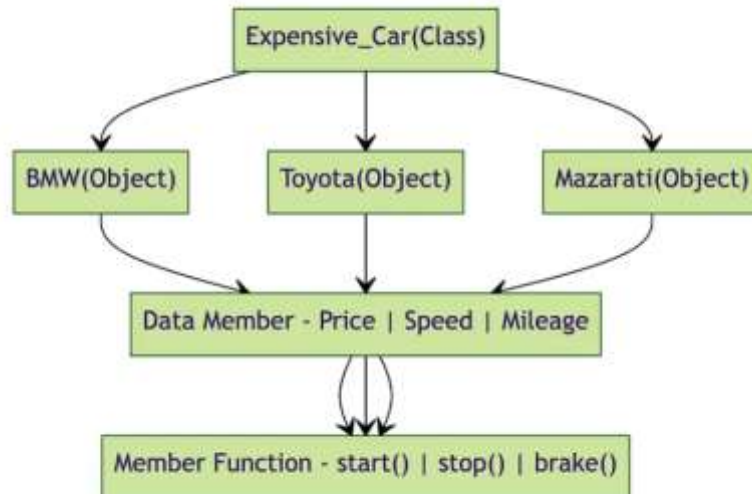
Example

Class: Expensive Cars

Objects: Mercedes, BMW, Toyota

Data Member: Price, Speed, Wheels, Mileage, Model, Color

Member Function: brakes(), speed(), start(), stop(), etc.



Class

Syntax:

```
class ClassName {  
    access_specifier:  
    // Data_members;  
    // Member_Functions()  
};
```

- The **keyword class** is used to define a class in C++, where the class identifier is given by **ClassName**.
- The **access_specifier** is the access controller which defines the visibility of class members. It can be private, public, and protected; we will elaborate on these in a later section.
- **Data_members** are the variables that hold data related to an object of the class.
- **Member_functions()** are class methods that define what operations can be performed on objects and how.

Understanding Objects in C++

*An **object** is anything in the real world that has a distinct identity, observable characteristics, and defined behaviors.*

Think of it like a **smartphone**:

State (Attributes): Brand, model, color, battery level.

Behavior (Methods): Making a call, sending a text, or opening an app.

According to the Webster's New World dictionary, an Object is defined as:

“A thing that can be seen or touched; material thing; a person or thing to which action, thought or feeling is directed”.

An **object** is an instance of a class. When an object is created, a real-world entity is created that has the properties and behaviors defined by the class. While a class only defines what an object will be, an object is an actual instance that occupies memory and can be manipulated.

Understanding Objects in C++

Creating an Object:

To use the data members and functions of a class, you need to create an object. When you create an object, memory is allocated for it, allowing you to access and manipulate the data members and functions defined in the class.

Syntax:

```
ClassName ObjectName; //Creating an object
```

- **Class_name:** It is the name of the class for which the object is created
- **Object_name:** It is the name of the object. Note that having a meaningful object name enhances code readability

Example:

```
Car myCar; // Creating an object of the Car class
```

Creating Class & Object in C++

- 1. Declare and Define a Class in C++ Program:** The first step is to create a class, as mentioned in the syntax given above. The C++ class definition must specify-
 - ✓ Data members, i.e., the attributes or member variables of objects in C++.
 - ✓ Member functions, i.e., C++ class methods that define how we can manipulate the data stored in the member variables to full required tasks.
- 2. Create an Object in C++ Program:** After a class has been defined, an object of that class can be declared by writing the class name and then the object name. The memory for the object is assigned after this declaration.
- 3.** The creation of a C++ class object is usually done in the main() part of the program.
- 4.** Once declared and initialized, the C++ class object can be used to invoke special functions in the class.

Class, Object and Member

```
#include <iostream>
using namespace std;
// Define a class called 'Car'

class Car {
    public:        // Access specifier
        string brand; // Data member
        int speed;    // Data member

        void display() {        // Member function
            cout << "Brand: " << brand << ", Speed: " << speed << " km/h" << endl;
        }
};

int main() {
    // Creating an object of the Car class
    Car myCar;

    // Setting values for the object's data members
    myCar.brand = "Toyota"; // Assign brand name
    myCar.speed = 120;      // Assign speed value

    // Displaying the car details
    myCar.display(); // Call the display function to output car information

    return 0; // Indicate successful program completion
}
```

Data Member in C++

A **data member** is a variable declared within a class or struct that holds data (state) for objects of that class type. It represents the attributes or properties of the class.

Formal Definition

A data member (also called a field, attribute, or instance variable) is a non-function member of a class that stores data. It can be of any type, including fundamental types, pointers, references, arrays, or other class types. Each data member has storage associated with it for each object instance (except static members, which are shared).

Data Member in C++

```
#include <iostream>
class ClassName {
    // Data member declaration syntax:
    [access_specifier:] [modifiers] type name [= initializer];

private:
    int age;                // simple data member
    std::string name;       // object data member
    static int count;       // static data member
    const double PI = 3.14159; // const data member
    int* ptr;               // pointer data member
    int& ref;               // reference data member (must be initialized)
    mutable int cache;      // mutable data member
    int arr[10];            // array data member
    std::vector<int> scores; // container data member
};
```

Data Member in C++

Types of Data Members

Type	Declaration	Storage	Example
Instance	type name;	Per object	int x;
Static	static type name;	Shared across all objects	static int count;
Const	const type name;	Per object, cannot modify	const int id;
Reference	type& name;	Per object, must bind	int& ref;
Mutable	mutable type name;	Can modify in const methods	mutable int cache;
Bit-field	type name : bits;	Space-efficient	int flag : 1;

Member Function in C++

A function that is defined as a class member is known as a member function in C++ programming.

What Is Member Function In C++?

- **Member functions** are functions defined within a class.
- They can access the values of the data members and can also carry out operations on the data members.
- Their definition may be included in or excluded from the class body. A member function in C++ can access both public and private class members.

Member Function in C++

Syntax for Member Function

```
// Member function declaration  
returnType functionName (arguments)  
{//function body  
};
```

- The **functionName** refers to the name of the function, with the **returnType** specifying the data type of its return value.
- The **arguments** refer to the parameters the member function can take.
- The **curly brackets {}** contain the function body, i.e., the operations that are performed on the data members of the class.

Member Function in C++

Additional points to note about a member function in C++ is that it can access the private data of members, whereas a non-member function is unable to do so.

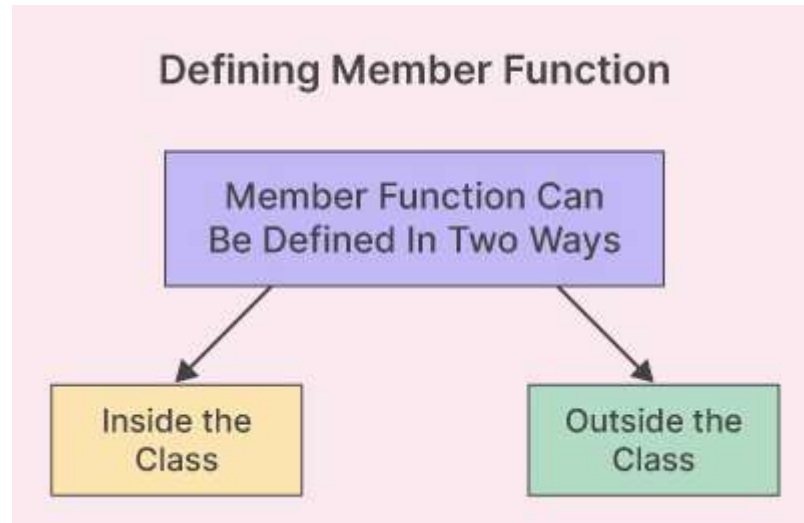
- Also, they can be called by another member function directly without using a dot operator.
- Member function makes the program modular by increasing code reusability and making it easier to understand, hence making the code maintainable.
- Member functions also provide the ability to restrict access to the data of members of the class. This can be done by making specific functions private, thus ensuring that only the class itself may access or modify sensitive data.
- Member functions are essential in C++ as they encapsulate the behavior and functionality of objects. They enable objects to interact with their data and serve as a means for other components of the program to modify the data.

Member Function in C++

Defining a Function in C++

There are mainly two ways to define a member function in C++. They are-

- Inside the class definition
- Outside the class definition.



Member Function in C++

Defining a Function in C++ Inside The Class Definition

A member function can be both declared and defined within a class itself. When a member function is defined within a class, it is referred to as an inline function. All of the constraints and limitations that apply to inline functions also apply to functions declared within a class.

Syntax:

```
class MyClass{  
    private:  
        //Data Members variables  
  
    public:  
        //Member Function  
        returnType functionName(argument){  
            //Function Definition  
        }  
};
```

Member Function in C++ Inside The Class Definition

```
#include <iostream>
using namespace std;

class Circle{
private:
//Data Members
float rad;

public:
//Member Function
void getradius(){
cout<<"Enter Radius"<<endl;
cin>>rad;}

void calc_area(){
float a;
a = 3.14*rad*rad;
cout<<"The Area of the Circle is = "<<a<<endl;
};

int main(){
    Circle c;
    c.getradius();
    c.calc_area();
    return 0;
}
```

- *Circle class* is defined , which has a *private data member* called *rad* (of float type).
- In the class is the member function called *getradius()*, which prompts the user to enter the radius of the circle *using cout*, stores it in the rad data member, and reads the input using the *cin command*.
- Also, another void member function called *calc_area* within the Circle class. It calculates the area of the circle using the formula $a = 3.14 * rad * rad$ and displays it on the console.
- In the *main() function*, we first create an *object c* of the Circle class.
- We then *call the getradius()* member function on the c object, which prompts the user for input and stores the radius in the rad data member. The *calc_area()* member function then calculates the area of the circle and displays it on the console.

Member Function in C++

Defining a Function in C++ Outside The Class Definition

In this case, the declaration and definition of the member function are not made together. That is, while the functions are declared inside the class, their definition is given outside.

However, the definitions of member functions are very similar to those of normal functions. They should have a function header and body. Unlike a normal function, a member function includes a membership identity label in the header. The identity label is used to indicate that the function belongs to a particular class.

Member Function in C++

Syntax:

```
return_type class_name :: function-name (argument declaration){  
    //Function body  
}
```

Here,

- The **class_name** is the membership label or the name of the class to which the member function belongs.
- The **function_name** and the double colon (**::**) are the name of the function and the scope resolution operator, which tells the compiler that the function belongs to the class_name.
- The **arguments** and the **return_type** refer to the parameters the function takes and the data type of its return value, respectively.

Member Function in C++ Inside The Class Definition

```
#include <iostream>
using namespace std;

class Circle{
private:
float rad; //Data Members

public:
void getradius(); //Member Function
void calc_area(); //Member Function
};

void Circle :: getradius(){
cout<<"Enter Radius ";
cin>>rad;}

void Circle :: calc_area(){
float a;
a = 3.14*rad*rad;
cout<<"The Area of the Circle is = "<<a<<endl;}

int main(){
    Circle c;
    c.getradius();
    c.calc_area();
    return 0;
}
```

- *Circle class* is defined , which has a *private data member* called *rad* (of float type).
- It contains the member functions called *getradius()*, *calc_area()* which are defined outside the class using scope resolution operators (::)
- In the *main() function*, we first create an *object c* of the Circle class.
- We then *call the getradius()* member function on the c object, which prompts the user for input and stores the radius in the rad data member. The *calc_area()* member function then calculates the area of the circle and displays it on the console.

Static Data Member

Introduction

- Declared using the keyword `static` inside the class.
- Must be defined outside the class .
- Shared by all the objects of the class.
- Accessed using the class name.

A static data member in C++ refers to a member of a class that is common to all its objects. In other words, it is associated with the class as a whole. These members are created using the static keyword.

What Is Static Data Member In C++?

The static data member in C++ programming is a special type of class member that is associated with the class rather than with an individual object of a class. It means that all instances of the class share the same static data member. In other words, unlike regular (non-static members) data members, which have separate copies for each object, static data members are shared among all objects of the class.

Static Data Member

Syntax to Declare a Static Data Member In C++:

```
class ClassName {  
    access_specifier:  
    static data_type data_member_name;  
};
```

- The **ClassName** and **data_member_name** refer to the names of the class and the data member we are creating.
- An **access specifier** indicates the visibility of the class member, which can be private, public or protected.
- The **static** keyword indicates that the respective member is of a static nature and the type of data it stores is given by **data_type**.

```
Declaration:      Static datatype variableName:  
Definition :      datatype className:: variableName;  
Usage:            ClassName:: variableName;
```

By default, static data members are initialized by 0.

Static Data Member

Characteristic of Static Data Member

- ✓ **Declaration of Static Data Member in C++:** A static data member in C++ is declared using the static keyword within the class definition.
- ✓ **Definition of Static Data Member in C++:** The static data member in C++ must be defined outside the class (usually in the source file) to allocate memory for it. This is typically done by specifying the data type and the name of the static data member, along with its initial value (if any).
- ✓ **Initialization of Static Data Member in C++:** We can initialize a static data member in C++ programs using a constant expression or a compile-time constant. The initialization is done outside the class definition.
- ✓ **Accessing a Static Data Member in C++:** We must use the class name followed by the scope resolution operator(::) and the name of the respective static data member in C++ when we want to access it.

Class Work

Context: Student count using Static Data Member

Problem Statement

- 1. Needed a Program that will output the total number of students(object) recorded*
- 2. Also demonstrate the difference with normal Data Member*

Static Member Function

What Is Static Member function in C++?

Static member functions in C++ are class-level functions that are associated with the class itself rather than with individual instances (objects) of the class. They are declared using the static keyword, and we can call such a function with a class name without the need to create an object. Static member functions are commonly used for operations that are related to the class as a whole rather than to specific instances.

Static Member Function

Syntax to Declare a Static Data Member In C++:

```
class ClassName {  
    public:  
        static returnType functionName(parameters) {  
            // Function body implementation  
        }  
};
```

- The **static** keyword indicates that the function of the name- **functionName** is static in nature.
- The **parameters** and **returnType** refer to the input parameters (if any) taken by the static member function and the data type of its return value, respectively.
- As marked by the code comment, the **curly brackets** contain the body of the function.


```
#include <iostream>
using namespace std;

class MyClass {
public:
    static int staticValue; // Static data member declaration

    static int getStaticValue() {
        return staticValue; // Static member function to access
        the static data member
    }
};

// Initialization of the static data member outside the
class
int MyClass::staticValue = 101;

int main() {
    // Accessing the static data member using the static
    member function
    int value = MyClass::getStaticValue();
    std::cout << "Value of Static Data Member: " << value <<
    std::endl;

    return 0;
}
```

We define a class called ***MyClass*** and declare a static data member ***staticValue*** of type int. This static member is shared across all instances of the class.

We also declare a static member function ***getStaticValue()*** within the class.

This function returns the value of the static data member ***staticValue***.

Outside the class, we initialize the static data member ***staticValue* to 101.**

This initialization is necessary because static members must be defined outside the class to allocate storage.

In the ***main()* function**, we call the static member function ***getStaticValue()*** using the class name ***MyClass*** and the ***scope resolution operator (::)***.

This function call retrieves the value of ***staticValue***, which is ***101***. We finally store the returned value in an integer variable ***value*** and print it to the console.

Static Member Function – Usages

Static member functions in C++ serve several important purposes and offer distinct advantages in various programming scenarios. Here are some key reasons why static member functions are needed and valuable in C++:

Class-Level Operations: Static member functions allow you to define operations that are relevant to the class as a whole rather than to individual instances (objects) of the class. This is especially useful when you want to perform actions that do not depend on the specific state or data of objects.

No Object Instantiation Means Efficient Code: You can call a static member function in C++ directly on the class without the need to create instances of the class. This means you can use these functions without incurring the overhead of object creation, which can lead to more efficient code execution.

Utility Functions: Static member functions are often used to encapsulate utility functions that provide common functionality related to the class. These utility functions can perform tasks such as mathematical calculations, data validation, and formatting, making your code more organized and reusable.

Static Member Function – Usages

Factory Methods: Static member functions in C++ are commonly employed as factory methods that create instances of a class with specific characteristics. Factory methods allow you to control the object creation process and provide a convenient way to initialize objects.

Singleton Pattern: Static member functions are crucial for implementing the singleton design pattern, which ensures that only one instance of a class exists throughout the application's lifetime. A static member function in C++ can create and manage a single instance, providing global access to that instance.

Global Access: Static member functions can be called from anywhere in your code by specifying the class name followed by the scope resolution operator (::). This makes them accessible globally, allowing you to perform operations that span multiple objects or modules.

Code Organization: By grouping related functions within a class and using a static member function in C++, you can improve code organization and maintainability. It becomes clear which functions are associated with a particular class, enhancing the structure of your codebase.

Property	Regular Member Function	Static Member Function
Association	Associated with instances of the class.	Associated with the class itself.
Access	Using object instances	Using the class name and scope resolution (::) operator
Can access static data members	Yes (via object or class name)	Yes (via class name and scope resolution operator)
Can access non-static data members	Yes (via object)	No (unless passed as a parameter)
Can access regular member functions	Yes (via object)	No
Can access static member functions	Yes (via object or class name)	Yes (via class name and scope resolution operator)
Access to 'this' pointer	Yes	No
Lifetime	Tied to the object's lifetime.	Tied to the class's lifetime.
Initialization	Per instance	Typically, at class declaration
Virtual Function	Can be declared virtual	Cannot be declared virtual
Polymorphism	Participates in polymorphism	Does not participate in polymorphism

Accessing Class Members

Members of the class can be accessed inside the class itself simply by using their assigned name and access them outside, the (.) dot operator is used with the object of the class.

Syntax:

```
object.dataMember;  
object.memberFunction();
```

```
#include <iostream>  
using namespace std;  
  
class Dog {  
    public:  
        string name;  
        string breed;  
  
        void display(){  
            cout << "Dog Name: " << name << endl;  
            cout << "Breed: " << breed << endl;  
        }  
};
```

```
int main() {  
    Dog tuffy;  
    tuffy.name = "Tuffy";  
    tuffy.breed = "Papillon";  
    tuffy.display();  
    return 0;  
}
```


Nesting of Members Function

We know that only the public members of a class can be accessed by the object of that class, using dot operator. However, a member function can call another member function of the same class directly without using the dot operator. This is called as *nesting of member functions*.

Characteristics of Nesting Members Function:

- Nesting of member functions occurs when one member function calls another within the same class.
- A member function can be called using the dot (.) operator with an object or directly by other member functions of the same class.
- All member functions can access and call other member functions within the class.
- Static member functions can be called by other member functions but can only directly call other static member functions.
- Member functions can be recursive, meaning they can call themselves.
- Member functions defined outside the class can also exhibit nesting, adhering to accessibility and scope rules.

Nesting of Members Function

```
#include <iostream>
using namespace std;

class c1
{
    int s = 2;
public:
    void m1() {
        cout << "m1 ";
    }
    void m2()
    {
        // Non-static member function calling another non-static member function
        m1();
        cout << "m2 ";
    }
};

int main()
{
    c1 obj;
    obj.m2();
}
```

Nesting of Members Function

```
#include <iostream>
using namespace std;
class set{
    int m,n;

    public:
        void input();
        void display ();
        int largest();
};
```

```
int set::largest(){
    if(m>n)
        return m;
    else
        return n;
}
```

```
void set::input(){
    cout<<"input values of m:";
    cin>>m;
    cout<<"input values of n:";
    cin>>n;
}

void set::display(){
    cout<<"largestvalue="<<largest()<<"\n";
}
```

```
int main(){
    set A;
    A.input();
    A.display();
    return 0;
}
```

Nesting of Members Function

```
#include <iostream>
using namespace std;

class average{
    int a,b;
public:
    void read();
    void print();
    int avg();
};

void average::read(){
    cout<<"\n enter a and b: ";
    cin>>a>>b;
}

void average::print(){
    cout<<"value of a: "<<a;
    cout<<"\nvalue of b: "<<b;
    cout<<"\naverage is : "<<avg();
}

int average::avg(){
    return (a+b)/2;
}

int main(){
    average A;
    A.read();
    A.print();
    return 0;
}
```

this pointer

The 'this' pointer in C++ is an implicit pointer available within non-static member functions of a class or structure. It points to the current object instance, letting the object access its own member variables and functions.

Syntax of 'this' Pointer

When creating or using the 'this' pointer, the 'this' keyword is used in conjunction with the arrow operator (->) along with the name of the member or method being referred to.

```
void functionName() {  
    this->memberName = value;  
}
```

- ✓ **functionName** refers to the identifier/ name you have given to the function.
- ✓ The term **this->** represents the 'this' pointer in C++, and **memberName** refers to the respective member of the class you are trying to access.

this pointer

Syntax of 'this' Pointer

```
class MyClass {  
    public:  
        void myMethod() {  
            this->myMember = 42;  
        }  
    private:  
        int myMember;  
};
```

In the above code snippet, we are trying to access the myMember variable/ attribute of the current object through the expression **this->myMember**, which is being assigned a value of 42 within the **myMethod()** function definitio

this pointer

Defining 'this' Pointer In C++

In C++ programming, each object of a class has its own set of member variables/ attributes (also known as data members) and member functions.

- When a member function is called on an object, the 'this' pointer is automatically passed as a hidden argument to the member function.
- It enables the member function to have access to the object's data members (i.e., the class member variable) and call other member functions.

```
#include <iostream>
using namespace std;

class Person {
private:
    string name;
    int age;
public:
    Person(const std::string& name, int age) {
        this->name = name;
        this->age = age;
    }
    void displayInfo() {
        std::cout << "Name: " << this->name << std::endl;
        std::cout << "Age: " << this->age << std::endl;
    }
    void updateAge(int newAge) {
        this->age = newAge;
    }
};

int main() {

    Person person("Kapil", 19);
    person.displayInfo();
    person.updateAge(20);
    cout << "After updating age:" << endl;
    person.displayInfo();
    return 0;
}
```


Analogy & PU Question



Real-World Analogy

The 'this' pointer is like saying 'myself'. When an employee says 'I will handle this myself', 'myself' refers to that specific person — just as 'this' refers to the specific object calling the method. Static members are like a shared office kitchen — all employees (objects) share one, not one per person.



Probable PU Exam Question

Q: Explain static data members and static member functions in C++. Write a program to count objects using a static variable. Explain the 'this' pointer. [PU Exam 2022]

Access Specifier / Modifier

Introduction

- Keywords that define level of accessibility of members of a class.
- Controls who can access what in a program

When designing a C++ class, we often want to control who all can access members (both the data members and member functions) of a class.

Access specifiers are a way to achieve this. Creators of a class can use access specifier to implement data hiding. Data Hiding helps in separating implementation from interface.

Type of Access Specifiers:

- Public
- Private
- Protected



Access Specifier : Public

Public Access Specifier

Members are accessible from anywhere in the program, both inside and outside the class.

Purpose: Defines the class's public interface, allowing unrestricted access to members. This is how users (other classes, functions, or main) interact with the class.

Behavior in Inheritance: Public members remain public in derived classes under public inheritance, making them accessible to both derived classes and external code.

Use Case: Public methods for operations like initialization, data retrieval, or actions (e.g., `start()`, `stop()`).

Access Specifier : Public

```
#include <iostream>
using namespace std;
// Define a class called 'Car'

class Student {
    public:          // Access specifier
        string name; // Data member
        int rollNo;  // Data member

        void display() {          // Member function
            cout << "Name: " << name << endl;
            cout << "RollNo: " << rollNo << endl;
        }
};

int main() {
    // Creating an object of the Car class
    Student s1;

    s1.name = "Ankit";
    s1.rollNo = 2;
    s1.display();
    return 0; // Indicate successful program completion
}
```

Access Specifier : Private

Private Access Specifier

Accessible only within the class. Not accessible to derived classes or external code.

- **Purpose:** Ensures strict encapsulation by limiting access to the class's internal implementation. This protects data integrity and prevents misuse by external code or derived classes.
- **Behavior in Inheritance:** Private members are **not inherited** by derived classes, meaning they cannot be accessed directly or indirectly in subclasses.
- **Use Case:** Use private for sensitive data (e.g., internal counters, buffers) or helper methods that shouldn't be exposed.

Access Specifier : Private

```
#include <iostream>
using namespace std;
class BankAccount {
    private:
        double balance; // Private data member
    public:
        BankAccount(double initialBalance) {
            balance = initialBalance;
        }

        double getBalance() { // Public function to access private data
            return balance;
        }
};

int main() {
    BankAccount account(1000.0);
    // account.balance = 500; // Error: Can't access private member directly
    cout << "Balance: $" << account.getBalance() << endl;
    return 0;
}
```

Access Specifier : Protected

Protected Access Specifier

Accessible only within the class and in derived classes but not externally.

- **Purpose:** Allows derived classes to access base class members while still hiding them from external code. This is useful for inheritance hierarchies where subclasses need access to base class internals.
- **Behavior in Inheritance:** Protected members are inherited by derived classes and remain accessible unless the inheritance type restricts them further (e.g., private inheritance).
- **Use Case:** Base class members that derived classes need to manipulate (e.g., shared resources or state).

Access Specifier : Protected

```
#include <iostream>
using namespace std;
class Animal {

protected:
    string species; // Protected member
public:
    void setSpecies(string s) { species = s; }
};
class Dog : public Animal {
public:
void showSpecies() {
cout << "Species: " << species << endl; // Accessible in derived class
}
};
int main() {
    Dog myDog;
    myDog.setSpecies("XYZ");
    myDog.showSpecies();
    return 0;
}
```


Access Specifier

Common Pitfalls and Best Practices

1. Overusing Public Members:

Making too many members public can expose implementation details, making the class harder to maintain or refactor.

Solution: Use private/protected members with public getters/setters for controlled access.

2. Misunderstanding Protected:

Developers sometimes assume protected members are accessible outside the class hierarchy, but they're not.

Solution: Clearly document which members are intended for derived classes.

3. Inheritance Misuse:

Using private inheritance when composition would be better can lead to complex code.

Solution: Prefer composition (has-a relationship) over inheritance (is-a relationship) unless inheritance is clearly justified.

4. Performance Considerations:

Access specifiers don't directly impact performance, but excessive use of getters/setters for private members can add overhead in performance-critical code.

Solution: For performance-sensitive code, consider inline getters or direct access (if safe).

Array of Object

An array of objects is a collection of multiple objects of the same class type stored contiguously in memory. It allows you to manage multiple instances of a class efficiently using array indexing and iteration.

Arrays can store primitive data types like int, float, double, char, and even derived data types such as structures and pointers.

When a class is defined, it specifies the structure of objects but does not allocate memory. To use the data and functions defined in the class, you need to create objects. An array of a class type is known as an **array of objects**.

Basic Syntax

```
ClassName arrayName[arraySize];
```

Array of Object

Key Points for Beginners:

- **Declaration:** `ClassName arrayName[size];`
- **Access:** Use `arrayName[index].member` to access object members
- **Indexing:** Starts from 0, ends at size-1
- **Loop:** Use for loop to process all objects
- **Constructor:** Can initialize objects while creating the array

Most Common Simple Use Cases:

- **Student database:** Store student records
- **Product inventory:** Store product details
- **Employee list:** Store employee information
- **Game objects:** Store game characters or items
- **Book library:** Store book information

Array of Object

```
#include <iostream>
using namespace std;

class student
{
    char name[20];
    int age;
public:
    void getdata() {
        cout<<"enter the name of the student :"<<endl;
        cin>>name;
        cout<<"enter the age :"<<endl;
        cin>>age;
    }
    void putdata(){
        cout<<"the name of the student is :"<<name<<endl;
        cout<<"the age of the student is :"<<age<<endl;
    }
};
```

```
int main(){
    student s[10];
    int n;
    cout<<"Enter the number of
students :"<<endl;
    cin>>n;
    for (int i=0;i<n;i++) {
        s[i].getdata();
    }
    for (int i=0;i<n;i++) {
        s[i].putdata();
    }
    return 0;
}
```

Friend Function — Key Concepts

- NOT a member of the class but has access to private/protected data
- Declared inside class using 'friend' keyword
- Called without dot notation — object is passed as parameter
- Has NO 'this' pointer
- Does NOT inherit friendship
- Use cases:
- Operator overloading (especially <<, >>)
- Functions that need access to two different class private members
- Bridge between two unrelated classes

```
class Distance {  
    float km;  
    public:  
        Distance(float k): km(k) {}  
        // Friend declaration  
        friend Distance add(  
            Distance d1, Distance d2);  
        void show() { cout<<km; }  
};  
  
// NOT a member — no Distance::  
Distance add(Distance d1, Distance d2){  
    return Distance(d1.km + d2.km);  
}
```

Friend Function

A **friend function** is a non-member function that has access to the private and protected members of a class. It is declared inside the class using the friend keyword but defined outside the class scope.

Key Characteristics

- **Not a member** of the class
- **Can access private/protected** members directly
- **Declared with** friend keyword inside the class
- **Defined outside** the class like a normal function
- **Can be a function** or a member function of another class

Friend Function

Basic Syntax

```
class ClassName {  
private:  
    int privateData;  
  
public:  
    // Declaration of friend function  
    friend returnType functionName(parameters);  
};  
  
// Definition of friend function (outside the class)  
returnType functionName(parameters) {  
    // Can access private members of ClassName  
}
```

Friend Function

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width;
    int height;
public:
    Rectangle(int w, int h) : width(w), height(h) {}

    friend void displayArea(Rectangle r); // Declare friend function
};

// Friend function definition - can access private members
void displayArea(Rectangle r) {
    // Accessing private members directly
    int area = r.width * r.height;
    cout << "Area: " << area << endl;
}

int main() {
    Rectangle rect(5, 10);
    displayArea(rect); // Called like normal function
    return 0;
}
```


Analogy & PU Question



Real-World Analogy

A friend function is like a trusted accountant who is NOT an employee of your company but is allowed to see private financial records. They work outside the company but have special access granted by the company (class) itself.



Probable PU Exam Question

Q: What is a friend function? Differentiate it from a member function. Write a C++ program to add two distance objects using a friend function. [PU Exam 2021]

Friend Class

A **friend class** is a class that is granted access to the private and protected members of another class. When Class A declares Class B as a friend, all member functions of Class B can access the private and protected members of Class A.

Key Characteristics

- **Bidirectional?** No, friendship is **not mutual** (unless explicitly declared both ways)
- **Not inherited:** Friendship is not passed to derived classes
- **Not transitive:** If A is friend of B, and B is friend of C, A is NOT friend of C
- **Can be declared** in public, private, or protected sections (convention: public)
- **Forward declaration** may be needed

Friend Class

Basic Syntax

```
// Forward declaration (if needed)
class FriendClass;

class MyClass {
private:
    int privateData;

public:
    // Declare FriendClass as friend
    friend class FriendClass;
};

class FriendClass {
public:
    void accessMyClass(MyClass& obj) {
        // Can access privateData directly
        obj.privateData = 100;
    }
};
```

Friend Class

Friend classes are useful when:

- Two classes are tightly coupled (e.g., Engine and Car)
- Implementing design patterns (Factory, Builder, Iterator)
- Creating test classes
- Implementing data access layers
- Performance optimization is critical

Comparison: Friend Class vs Friend Function

Aspect	Friend Class	Friend Function
Scope	All member functions are friends	Only specific function
Granularity	Coarse (entire class)	Fine (single function)
Use case	Related classes working together	Helper/utility functions
Coupling	Creates tighter coupling	Less coupling
Complexity	Higher	Lower

Passing and Returning Objects in C++

In C++, objects can be passed to functions and returned from functions just like primitive data types. However, understanding the different ways to pass and return objects is crucial for writing efficient and bug-free code.

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width;
    int height;

public:
    Rectangle(int w = 0, int h = 0) : width(w), height(h) {
        cout << "Constructor called" << endl;
    }

    Rectangle(const Rectangle& other) {
        width = other.width;
        height = other.height;
        cout << "Copy constructor called!" << endl;
    }

    int area() const {
        return width * height;
    }

    void setWidth(int w) {
        width = w;
    }
};
```

Passing Objects to Functions

Pass by Value (Copy): The object is copied when passed to the function. The function works with a copy, so the original object is not modified.

```
int main() {
    Rectangle rect(5, 10);
    cout << "Original area: " << rect.area() << endl;

    modifyRectangleByValue(rect); // Copy is created

    cout << "Original area after function: " << rect.area() << endl;
    // Original remains unchanged

    return 0;
}
```

Passing and Returning Objects in C++

Passing Objects to

Functions

Pass by Reference:
The function receives a reference to the original object. No copy is created, and changes affect the original.

```
#include <iostream>
using namespace std;

class Counter {
private:
    int count;

public:
    Counter(int c = 0) : count(c) {}

    void increment() {
        count++;
    }

    int getCount() const {
        return count;
    }
};

// Pass by reference - no copy, modifies original
void incrementByReference(Counter& c) {
    c.increment();
    cout << "Inside function - count: " << c.getCount() << endl;
}
```

```
int main() {
    Counter counter(5);
    cout << "Original count: " << counter.getCount() << endl;

    incrementByReference(counter); // No copy made

    cout << "Original count after function: " << counter.getCount() << endl;
    // Original is modified

    return 0;
}
```

Passing and Returning Objects in C++

Passing Objects to

Functions

Const Reference:
The function receives a read-only reference. No copy is created, but the object cannot be modified

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;
    int rollNo;

public:
    Student(string n, int r) : name(n), rollNo(r) {}

    void display() const {
        cout << "Name: " << name << ", Roll: " << rollNo << endl;
    }

    string getName() const { return name; }
    int getRollNo() const { return rollNo; }
};

// Pass by const reference - read-only, efficient
void printStudent(const Student& s) {
    // s.setName("John"); // ERROR! Cannot modify const reference
    s.display(); // OK - const method
    cout << "Name from const ref: " << s.getName() << endl;
}

int main() {
    Student student("Alice", 101);
    printStudent(student); // No copy, but read-only

    return 0;
}
```


Passing and Returning Objects in C++

Returning Objects to Functions

1. Return by Value

The function returns a copy of the object

```
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;

public:
    Point(int a = 0, int b = 0) : x(a), y(b) {
        cout << "Constructor called" << endl;
    }

    Point(const Point& other) {
        x = other.x;
        y = other.y;
        cout << "Copy constructor called" << endl;
    }

    void display() const {
        cout << "(" << x << ", " << y << ")" << endl;
    }

    void setX(int val) { x = val; }
};
```

```
// Return by value - returns a copy
Point createPoint(int a, int b) {
    Point p(a, b); // Constructor called
    return p;      // Copy constructor called (or move in C++11)
}

int main() {
    cout << "Creating point using function:" << endl;
    Point p1 = createPoint(5, 10); // Copy constructor called (may be optimized)

    cout << "p1: ";
    p1.display();

    // RVO (Return Value Optimization) may eliminate copy
    cout << "\nWith RVO (may not show copy):" << endl;
    Point p2 = createPoint(20, 30);
    p2.display();

    return 0;
}
```

Passing and Returning Objects in C++

Returning Objects to Functions

3. Return by Reference

Returns a reference to an existing object. Must ensure the object outlives the function.

```
#include <iostream>
#include <vector>
using namespace std;

class Team {
private:
    string name;
    vector<string> players;

public:
    Team(string n) : name(n) {}

    void addPlayer(string player) {
        players.push_back(player);
    }

    // Return reference to internal data
    vector<string>& getPlayers() {
        return players; // Return by reference
    }

    void display() const {
        cout << "Team: " << name << endl;
        for(const auto& p : players) {
            cout << "    - " << p << endl;
        }
    }
};
```

```
// Return reference to an element
string& getFirstPlayer(Team& team) {
    return team.getPlayers()[0]; // Returns reference to player
}

int main() {
    Team team("Warriors");
    team.addPlayer("Curry");
    team.addPlayer("Thompson");
    team.addPlayer("Green");

    cout << "Original team:" << endl;
    team.display();

    // Get reference to first player and modify
    string& firstPlayer = getFirstPlayer(team);
    firstPlayer = "Durant"; // Modifies the original string

    cout << "\nAfter modification:" << endl;
    team.display();

    return 0;
}
```

Passing and Returning Objects in C++

Returning Objects to Functions

3. Return by Const Reference

Returns a read-only reference

```
#include <iostream>
#include <string>
using namespace std;

class Configuration {
private:
    string setting;
    static Configuration* instance;

public:
    Configuration() : setting("default") {}

    void setSetting(string s) { setting = s; }

    // Return const reference - read-only access
    const string& getSetting() const {
        return setting;
    }
};
```

```
int main() {
    Configuration config;
    config.setSetting("High Performance");

    const string& setting = config.getSetting();
    cout << "Setting: " << setting << endl;

    // setting = "New"; // ERROR! Cannot modify const reference

    // To modify, must use setter
    config.setSetting("Low Power");
    cout << "New setting: " << setting << endl; // Reflects change

    return 0;
}
```

Constructor and Destructor

- Characteristics of Constructors
- Default Constructor
- Parameterized Constructor
- Copy Constructor — deep copy vs shallow copy
- Constructor Overloading
- Destructor — cleanup on object death

Constructor

What Is Constructor In C++?

A constructor in [C++ programming](#) is a special member function that is called automatically when an object belonging to the respective class is created. It is the responsibility for the constructors to initialize the [object's data members](#) and establish its initial state.

- They make sure that an object is created with valid and consistent values, establishing the foundation for the object's functionality and behavior.
- Constructors in C++ are particularly helpful for allocating resources, setting initial values, and setting up the object for use immediately after creation.
- They have the same name as the class name and don't have the return type, not even void.

Syntax

```
class ClassName {  
    Access_specifier:  
    ClassName() {  
    }  
};
```

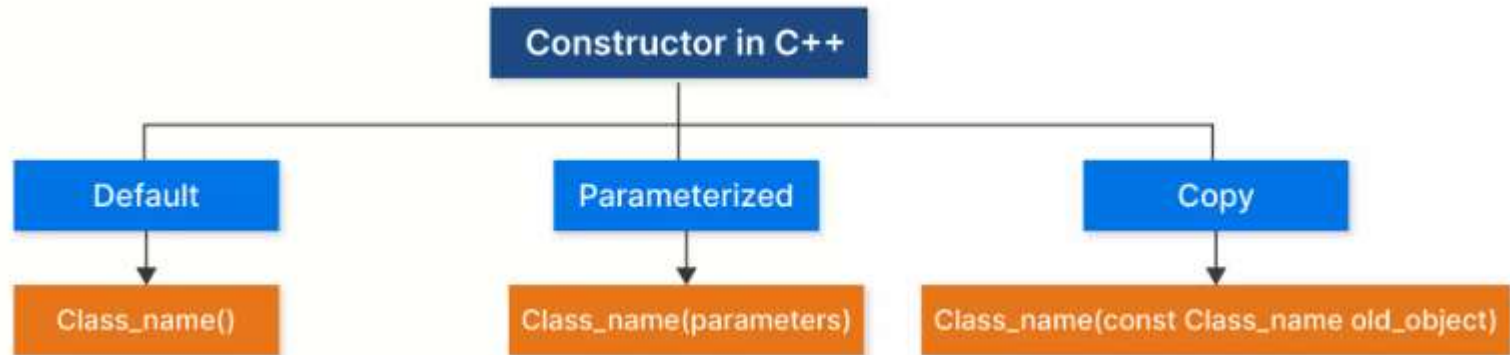
```
class Person {  
private:  
    string name;  
    int age;  
  
public:  
    // Default constructor  
    Person() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // Parameterized constructor  
    Person(const string& name, int age) {  
        this->name = name;  
        this->age = age;  
    }  
  
    // Member function to display person's information  
    void display() const {  
        cout << "Name: " << name << ", Age: " << age << endl;  
    }  
};  
  
int main() {  
    // Creating instances using constructors  
    Person person1; // Using default constructor  
    Person person2("Unstop", 7); // Using parameterized constructor  
  
    // Displaying person information  
    cout << "Person 1: ";  
    person1.display();  
    cout << "Person 2: ";  
    person2.display();  
  
    return 0;  
}
```

Constructor

A constructor in C++ plays a critical role in classes and the creation of objects. Here are some of its most important characteristics:

- **Class Name Matching:** A constructor's name must match the class name in which it is specified. It improves the compiler's ability to recognize and associate the constructor with the class.
- **Overloading:** We can overload constructors in C++, just like standard functions. This implies that a class may have numerous constructors, each with a unique list of parameters, providing various ways to initialize objects. It provides flexibility in code.
- **Default Constructor:** In case we do not explicitly define a constructor in C++ classes, the compiler creates a default constructor. The object's attributes are initialized with default values in the default constructor, such as 0 for numbers and null for object references.
- **No Return Type:** A constructor in C++ doesn't have any return types, not even void. This is because their primary function is to initialize objects instead of returning values.
- **Automatic Invocation:** When an object is formed, the constructors are automatically called. Unlike an ordinary method, a constructor in C++ does not require an explicit call.
- **Initialization:** An object's attributes, usually referred to as instance variables, are initialized using constructors. They guarantee that an item is in a usable state as soon as it is created.
- **Access Modifiers:** Access modifiers for constructors include public, private, and protected. The constructor in C++ can only be invoked from certain places depending on the access modifier selected.
- **Inheritance:** Unlike ordinary methods, constructors cannot be overwritten or inherited by subclasses.

Constructor



Default Constructor

The default constructor in C++ is a unique constructor that is created and called automatically by the compiler when no other user-defined constructor is explicitly declared within a class.

- The concept of default constructors is used to construct objects with initial default values for its data members and is called without any arguments.
- In other words, it is called for the default initialization of class internal members.

The default constructor in C++ also known as the argument constructor is, hence, crucial when creating objects without specifying any particular values.

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width;
    int height;

public:
    // Default constructor definition inside the class
    Rectangle() : width(0), height(0) {} // Using
    initializer list

    // Member function to display dimensions
    void displayDimensions() {
        cout << "Width: " << width << ", Height: " << height
        << endl;
    }
};

int main() {
    Rectangle rect; // Create a Rectangle object
    rect.displayDimensions(); // Display its dimensions
    return 0;
}
```


Parameterized Constructor

A **parameterized constructor** in C++ is a special member function that **accepts arguments to initialize an object with specific values at the exact moment of its creation.**

Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

- *The following class have brand, model and year attributes, and a constructor with different parameters. Inside the constructor we set the attributes equal to the constructor parameters (brand=x, etc).*

When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same

```
class Car {           // The class
public:               // Access specifier
    string brand;     // Attribute
    string model;     // Attribute
    int year;         // Attribute
    Car(string x, string y, int z) { // Constructor with
parameters
        brand = x;
        model = y;
        year = z;
    }
};

int main() {
    // Create Car objects and call the constructor with
different values
    Car carObj1("BMW", "X5", 1999);
    Car carObj2("Ford", "Mustang", 1969);

    // Print values
    cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";
    cout << carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";
    return 0;
}
```

Copy Constructor

A **copy constructor** is a special constructor in C++ that **makes a copy of an existing object while creating a new one.**

It is utilized whenever an object declaration is passed as a value to a function, returned by value from a function, or when an object is given the value of another object belonging to the same class.

The copy constructor in C++ makes sure that the new object's data members are set to the same values as those of the source object.

```
class Person {
private:
    string name;
    int age;

public:
    // Parameterized constructor
    Person(string n, int a) {
        name = n;
        age = a;}

    // Copy constructor
    Person(const Person &source) {
        name = source.name;
        age = source.age;}

    void displayInfo() {
        cout << "Name: " << name << ", Age: " << age << endl;}

};

int main() {
    Person person1("Shivani", 25);
    Person person2 = person1; // Copy constructor is called
    person2.displayInfo();

    return 0;
}
```

Copy Constructor

```
class Point {  
    public:  
        int x, y;  
        Point(int a, int b): x(a), y(b) {}  
  
        // Copy Constructor  
        Point(const Point &p) {  
            x = p.x; y = p.y;  
            cout << "Copy ctor called" << endl;  
        }  
        ~Point() { cout << "Destroyed" << endl; }  
};  
  
int main() {  
    Point p1(3, 4);  
    Point p2 = p1;    // Copy ctor called  
    Point p3(p1);    // Also copy ctor  
}
```

💡 If a class has dynamic memory, always define copy constructor to avoid shallow copy bugs.

Copy Constructor

What Happens Without a Copy Constructor?

If you don't define a copy constructor, C++ provides a **default copy constructor** that performs **shallow copy**:

For simple classes like this, the default works fine. But for classes with **dynamically allocated memory**, you **MUST** write your own copy constructor to avoid **shallow copy issues**.

```
// Default copy constructor (provided by compiler)
Rectangle(const Rectangle& other) {
    width = other.width;    // Copies value
    height = other.height;  // Copies value
    // No print message!
}
```

Copy Constructor

Shallow Copy vs Deep Copy

Concept	What Happens	Issue?
Shallow Copy	Copies pointer value, not the data it points to	Both objects point to same memory → Double delete!
Deep Copy	Creates new memory and copies the actual data	Safe - each object has its own memory

Constructor Overloading

Overloading is the technique of constructing numerous functions with the same name but various parameter lists. This is also application on constructor in C++.

Constructor overloading allows you to have multiple constructors in a class, each with a different parameter list. This enables objects to be initialized in various ways based on the provided arguments.

```
class Person {
private:
    string name;
    int age;

public:
    // Parameterized constructor
    Person(string n, int a) {
        name = n;
        age = a;}

    // Copy constructor
    Person(const Person &source) {
        name = source.name;
        age = source.age;}

    void displayInfo() {
        cout << "Name: " << name << ", Age: " << age << endl;}

};

int main() {
    Person person1("Shivani", 25);
    Person person2 = person1; // Copy constructor is called
    person2.displayInfo();

    return 0;
}
```

Constructor Overloading

```
class ClassName {  
access_specifier:  
// Default constructor  
ClassName() {}  
// Parameterized constructor with 1 parameter  
ClassName(parameter_type parameter1) {}  
// Parameterized constructor with 2 parameters  
ClassName(parameter_type parameter1, parameter_type parameter2) {}  
};
```

Here in Syntax,

- The **class** keyword is used to define the class.
- **className** is the name of the class in which the constructor is defined, as well as the name of all constructors, i.e., default and parameterized.
- The **access_specifier** specifies the visibility of the data members and member functions in the class.
- The term **parameter** refers to the parameter taken by the constructor. And **parameter_type** refers to their data type.
- **className(parameter_type parameter1)**: It is a parametrized constructor with 1 parameter.
- **className(parameter_type parameter1,parameter_type parameter2)**: It is a parametrized constructor with 2 parameters.

Constructor Overloading

```
#include <iostream>
#include <string>

using namespace std;

class Student {
public:
    // Default constructor
    Student() {
        name = "Unknown";
        age = 0;
    }

    // Parameterized constructor #1
    Student(string n) {
        name = n;
        age = 0;
    }

    // Parameterized constructor #2
    Student(string n, int a) {
        name = n;
        age = a;
    }
}
```

```
// Member function to display student information
void displayInfo() {
    cout << "Name: " << name << ", Age: " << age << endl;
}

private:
    string name;
    int age;
};

int main() {
    // Creating Student objects using different constructors
    Student student1; // Using the default constructor
    Student student2("Anya"); // Using parameterized
    constructor #1
    Student student3("Harsh", 20); // Using parameterized
    constructor #2

    // Displaying student information
    student1.displayInfo();
    student2.displayInfo();
    student3.displayInfo();

    return 0;
}
```


Destructor

A **destructor** in C++ is a special member function invoked when an object is destroyed, either when it goes out of scope or is explicitly deleted. It has the same name as the class, prefixed with a tilde (~).

- The C++ runtime automatically calls the proper destructor to release any resources retained by an object when its lifetime ends.
- This could be either because the object exits scope or if it is explicitly removed using the delete keyword (in the case of objects created dynamically).
- This ensures appropriate dynamic memory management to prevent memory leaks and resource wastage.

```
class FileHandler {
private:
    std::ofstream file;

public:
    FileHandler(const std::string& filename) {
        file.open(filename);
        if (!file.is_open()) {
            std::cout << "Error opening file.\n";
        }

        ~FileHandler() {
            if (file.is_open()) {
                file.close();
                std::cout << "The file closed successfully.\n";
            }
            // Other member functions and data members for file
            // handling operations
        };

        int main() {
            FileHandler fileHandler("example.txt");
            // Perform file operations...
            // fileHandler's destructor will automatically be called
            // at the end of the scope,
            // ensuring the file is properly closed, and resources are
            // released.
            return 0;
        }
    }
```

Constructor vs Destructor

Constructor

- Called when object is CREATED
- Can be overloaded
- Can have parameters
- Initializes object's data members
- Syntax: `ClassName()`
- Multiple constructors possible

Destructor

- Called when object is DESTROYED
- Cannot be overloaded
- No parameters
- Releases resources (memory, files)
- Syntax: `~ClassName()`
- Only ONE destructor per class

Analogy & PU Question



Real-World Analogy

A constructor is like hotel check-in: keys given, room configured. The destructor is checkout: return keys, clean room, free space for next guest. The copy constructor is like duplicating your hotel booking to give to a friend — a separate, independent reservation.



Probable PU Exam Question

Q: Explain all types of constructors in C++ with programs. What is a copy constructor and when is it invoked? [PU Exam 2023]

Operator Overloading

- What is Overloading
- Types of Overloading
- (Recap)Function Overloading
- Operator Overloading
- Rules of Operator Overloading
- Unary Operator Overloading (++ , -- , unary -)
- Binary Operator Overloading (+ , - , * , ==)
- Type Conversion: Object $\leftrightarrow$ Basic Type
- Overloading using Friend Functions

Overloading in C++

Creating two or more members that have the same name but are different in number or type of parameter is known as **overloading**.

In C++, we can overload:

- Methods
- Operators
- Constructors

We can overload these members because they have parameters *only*.

Types of overloading



Function Overloading

The process of having two or more functions with the same name, but different parameters, is known as **function overloading**.

The function is redefined by either using different types of arguments or a different number of arguments. It is only through these differences that a compiler can differentiate between functions.

The advantage of function overloading is that it increases the readability of a program because you don't need to use different names for the same action.

Different Ways of Function Overloading

A function in C++ can be overloaded in three different ways:

- By having different number of parameters.
- By having different types of parameters.
- By having both different number and types of parameters.

```
#include <iostream>
using namespace std;

class Cal {
public:
    static int add(int a,int b){
        return a + b;
    }
    static int add(int a, int b, int c) {
        return a + b + c;
    }
};

int main(void) {
    Cal C;          //class object declaration.
    cout<<C.add(10, 20)<<endl;
    cout<<C.add(12, 20, 23);
    return 0;
}
```

Function Overloading

Functions that Cannot be Overloaded

In C++, we have some specific scenarios in which the functions cannot be overloaded. As function overloading is primarily based on the function name and the types or number of its parameters. There could be cases where the C++ compiler cannot differentiate between functions and thus cannot be overloaded.

- Member function declarations with different access specifier but same name and same parameter-type-list cannot be overloaded.
- Functions with parameter declarations that differ only in a pointer * versus an array [].
- Functions with same parameter but different passing method (i.e. passed by value and passed by reference).
- Function with same parameter but different volatile type specifier.

Operator Overloading

Scenario:

Suppose we have two boxes:

Box A

5 Books

Box B

7 Books

We want to combine them.

Normally,

$$5 + 7 = 12$$

works because integers already know how to use the + operator.

But what if our Box itself is an object?

Box A + Box B

Does C++ know how to add two boxes? NO.

Because C++ only knows

int + int
float + float
double + double
char + char

It does NOT know

Box + Box
Student + Student
Time + Time
Distance + Distance
Money + Money

So, we teach C++ how to perform these operations.

This is called **Operator Overloading**

Operator Overloading

The concept of overloading a function can be applied to operators as well.

Real-Life Analogy:

Think of a universal remote control. The "Power" button works differently for your TV (turns it on/off), your sound system (switches between standby and active), and your air conditioner (turns cooling on/off). The same button (operator) performs different actions based on what device (data type) it's controlling.

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide special meaning to the user-defined data type. Operator overloading is used to overload or redefine most of the operators available in C++. It is used to perform the operation on the user-defined data type.

Technical Definition:

Operator overloading allows to define how operators (like +, -, *, etc.) work with user-defined data types (classes). It gives special meanings to existing operators when it is used with user-defined objects of the class.

Operator Overloading

Why Use Operator Overloading?

The primary reason for using operator overloading is to make code involving user-defined types as intuitive and readable as code involving built-in types.

For example,

if we have a Complex class to represent complex numbers, overloading the + operator allows us to write `c1 + c2` instead of `c1.add(c2)`.

This can make mathematical operations on complex numbers look more natural and familiar.



Operator Overloading

How Operator Overloading Works

In C++, operators are essentially functions with special names. When you overload an operator, you define a function that provides the operator's new behavior for specific types. The name of this function is formed by the operator keyword followed by the symbol of the operator being overloaded.

Operators can be overloaded in two primary ways:

- **Member Function:** The operator function is defined as a member of the class.
- **Non-Member Function:** The operator function is defined outside the class, usually as a friend function.

Operator Overloading

In C++, operator overloading is precisely that: defining a function with a special name—the ***operator*** keyword followed by the ***operator symbol***—to define custom behavior for that operator when applied to user-defined types.

- **Operators are function calls in disguise.** When the compiler sees `a + b`, it looks for a function named `operator+(a, b)` (or `a.operator+(b)` for a member function).
- **The name is fixed and mandatory.** The function name *must* be `operator@`, where `@` is the actual operator (e.g., `+`, `-`, `==`, `[]`, `()`).
- **You define the behavior entirely.** The function body determines what "addition," "equality," or "function call" means for your class.
- **Restrictions apply.** You cannot invent new symbols (e.g., `operator**`), nor can you overload operators for built-in types (e.g., redefining `int::operator+`).

Operator Overloading

Syntax

```
return_type operator  
symbol(parameters)  
{  
    ...  
}
```

- return_type = What the operation returns
- operator = Keyword (mandatory)
- symbol = The operator being overloaded (+, -, *, etc.)
- parameters = Operands (at least one must be user-defined type)

Example

```
Complex operator+(Complex obj)  
{  
    ...  
}
```

```
#include <iostream>  
using namespace std;  
  
class Complex {  
private:  
    int real, imag;  
public:  
    Complex(int r = 0, int i = 0) {real = r;    imag = i;}  
  
    // This is automatically called when '+' is used with  
    // between two Complex objects  
    Complex operator + (Complex obj) {  
        Complex temp;  
        temp.real = real + obj.real;  
        temp.imag = imag + obj.imag;  
        return temp;  
    }  
    void print() { cout << real << "+" << << imag << "i" << endl;  
    }  
};  
  
int main()  
{  
    Complex c1(10, 5), c2(2, 4);  
    Complex c3 = c1 + c2; // An example call to "operator+"  
    c3.print();  
    return 0;  
}
```

Operator Overloading

```
#include <iostream>
using namespace std;
class Shape{
private:
    int length;    // Length of a box
    int width;
public:
    // Constructor definition
    Shape(int l = 2, int w = 2){
        length = l;
        width = w;
    }
    double Area(){
        return length * width;
    }
    int operator + (Shape shapeIn){
        return Area() + shapeIn.Area();
    }
};

int main(void){
    Shape sh1(4, 4);    // Declare shapel
    Shape sh2(2, 6);    // Declare shape2

    int total = sh1 + sh2;
    cout << "\nsh1.Area() = " << sh1.Area();
    cout << "\nsh2.Area() = " << sh2.Area();
    cout << "\nTotal = " << total;
    return 0;
}
```

Operator Overloading Rules

- CANNOT overload: :: .* ?: sizeof
- At least one operand must be user-defined type
- Cannot change precedence or associativity
- Cannot create brand-new operators
- Cannot change number of operands
- Member function approach: left operand is the calling object
- Friend function approach: both operands are explicit parameters
- Use friend for commutative operations or when left operand is not your class (e.g., `cout << obj`)

```
class Complex {
public:
    float r, i;
    Complex(float a=0, float b=0):r(a),i(b){}

    // Binary + as member
    Complex operator+(Complex c) {
        return Complex(r+c.r, i+c.i);
    }

    // Unary - as member
    Complex operator-() {
        return Complex(-r, -i);
    }
};
```

Operator Overloading

Rules of Operator Overloading

1. **Syntax/Declaration Rule:** While overloading a function we need to use the keyword **operator** , **preceded by class name and followed by operator symbol such as '+'**.
2. **Member and Non-member functions:** We can overload operators in two ways:- first is as a member functions , second is as a non-member functions. When overloading as a member function, the left operand represents the object on which the function is called and when overloading as a non-member function, we can specify any type for the left operand.
3. **Number of operands Required:** Most operators can be overloaded with one or two operands. Example- Unary operators like ++ require one operand, while binary operators like + require two operands.
4. **Precedence and associativity:** Operator precedence and associativity cannot be changed when overloading operators. Example- It's not allowed to change the precedence of (*) or the associativity of (/).
5. **Return type:** When we overload operators than we must define the return type of the overloaded operator function according to our need.
6. **Friend function:** If overloaded operators needs to access private member variables/functions of a class than they must be declared as a friend function.
7. **Special Syntax:** For operators like (), [], or -> we must use a special syntax when overloading. Example, in order to overload the [] operator and to make objects callable like functions, we need to define a function named operator().

Operator Overloading

How Operator Overloading Works

In C++, operators are essentially functions with special names. When an operator is overloaded, we define a function that provides the operator's new behavior for specific types. The name of this function is formed by the operator keyword followed by the symbol of the operator being overloaded.

Operators can be overloaded in two primary ways:

- **Member Function:** The operator function is defined as a member of the class.
- **Non-Member Function:** The operator function is defined outside the class, usually as a friend function.

Operator Overloading Unary and Binary

BINARY

operator+

```
MyClass operator+  
(const MyClass&  
other)  
{  
    ...  
}
```

UNARY

operator-

```
MyClass  
operator-()  
{  
    ...  
}
```

Operators can be categorized as unary or binary based on the number of operands they take. Unary operators take one operand, while binary operators take two.

- **Unary Operators:** such as ++ (increment) and -- (decrement), operate on a single operand. To overload a unary operator, you define a member function with no parameters or a non-member function with one parameter.
- **Binary Operators:** such as +, -, *, and ==, operate on two operands. To overload a binary operator, you define a member function with one parameter or a non-member function with two parameters.

Operator Overloading Unary and Binary

What are Unary Operators?

Operators that work on ONE operand.

Examples: ++, --, !, ~, - (negative)

Real-Life Analogy:

The Light Switch - One switch (operator) affects one light (operand). You press it once, and it changes the state of that single light.

What are Binary Operators?

Operators that work on **TWO** operands.

Examples: +, -, *, /, ==, !=, <, >

Real-Life Analogy:

The Calculator - You input two numbers (operands), press the '+' button (operator), and get one result.

Operator Overloading Unary and Binary

```
#include <iostream>
using namespace std;

class Counter {
private:
    int value;

public:
    Counter(int v = 0) : value(v) {}

    // Prefix increment: ++obj
    Counter operator++() {
        ++value;          // Increment first
        return *this;     // Return updated object
    }

    // Postfix increment: obj++
    Counter operator++(int) { // int parameter is dummy for
postfix
        Counter temp = *this; // Save old state
        ++value;             // Increment
        return temp;         // Return old state
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};
```

```
void display() {
    cout << "Value: " << value << endl;
}

};

int main() {
    Counter c(5);

    cout << "Original: ";
    c.display(); // Output: 5

    Counter c1 = ++c; // Prefix
    cout << "After prefix ++: ";
    c.display();      // Output: 6
    cout << "Returned: ";
    c1.display();     // Output: 6

    Counter c2 = c++; // Postfix
    cout << "After postfix ++: ";
    c.display();      // Output: 7
    cout << "Returned: ";
    c2.display();     // Output: 6

    return 0;
}
```

Operator Overloading Unary and Binary

```
#include <iostream>
using namespace std;

class Complex {
private:
    float real, imag;
public:
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}

    Complex operator+(const Complex& c) {
        return Complex(real + c.real, imag + c.imag);
    }

    Complex operator-(const Complex& c) {
        return Complex(real - c.real, imag - c.imag);
    }

    bool operator==(const Complex& c) {
        return (real == c.real) && (imag == c.imag);
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};
```

```
int main() {
    Complex c1(3.0, 4.0); // 3 + 4i
    Complex c2(1.5, 2.5); // 1.5 + 2.5i

    Complex c3 = c1 + c2; // calls
    operator+(c2)
    c3.display(); // 4.5 + 6.5i

    Complex c4 = c1 - c2;
    c4.display(); // 1.5 + 1.5i
    cout << (c1 == c2) << endl; // 0 (false)
    return 0;
}
```

Operator Overloading – Type Conversion

When constants and variables of different types are mixed in an expression, C++ applies automatic type conversion to the operands as per certain rules. Similarly, an assignment operation also causes the automatic type conversion. The type of data to the right of an assignment operator is automatically converted to the type of variable on the left.

The three types of situations in the data conversion are:

1. Conversion from basic type to class type.
2. Conversion from class type to basic type.
3. Conversion from one class type to another class type

```
class Complex {  
    public:  
        float r, i;  
        Complex(float a=0, float b=0):r(a),i(b){}  
  
        // Binary + as member  
        Complex operator+(Complex c) {  
            return Complex(r+c.r, i+c.i);  
        }  
  
        // Unary - as member  
        Complex operator-() {  
            return Complex(-r, -i);  
        }  
};
```

Operator Overloading – Type Conversion

Basic Type to Class Type

To convert a primitive data type (like **int** or **double**) into a custom class object, we use a **parameterized constructor**. The constructor acts as the conversion mechanism.

```
#include <iostream>

class Distance {
private:
    int meters;
public:
    Distance() : meters(0) {}

    // Constructor handles: int -> Distance conversion
    Distance(int m) {
        meters = m;
    }

    void display() const {
        std::cout << meters << " meters\n";
    }
};

int main() {
    // Implicit conversion from basic type (int) to
    // Class type (Distance)
    Distance d = 50;
    d.display(); // Outputs: 50 meters
}
```

Operator Overloading – Type Conversion

Class Type to Basic Type

To convert a custom object back into a primitive data type, you must overload the **casting/conversion operator**.

Rules for Conversion Operators:

- It must be a **member function** of the class.
- It **cannot specify a return type** explicitly (the return type is implied by the operator's name).
- It takes **no arguments**.

```
#include <iostream>

class Fraction {
private:
    int num, den;
public:
    Fraction(int n, int d) : num(n), den(d) {}

    // Overloading the float cast operator: Class ->
    float
    operator float() const {
        return static_cast<float>(num) / den;
    }
};

int main() {
    Fraction f(3, 4);

    // Implicit conversion from Class (Fraction) to
    basic type (float)
    float val = f;
    std::cout << "Value: " << val << "\n";
    return 0;
}
```


Operator Overloading – Type Conversion

One Class Type to Another Class Type

Converting between two entirely different custom classes requires careful management to prevent code ambiguity. This type of conversion can be accomplished using two approaches

Approach A: Using a Constructor in the Destination Class

The destination class defines a constructor that accepts an instance of the source class as an argument.

```
class Inventory {
public:
    int items = 10;
};

// Destination Class
class Product {
private:
    int count;
public:
    Product() : count(0) {}

    // Constructor handles: Inventory -> Product
    conversion
    Product(const Inventory& inv) {
        count = inv.items;
    }
};
```

Operator Overloading – Type Conversion

Approach B: Using Conversion Operator in Source Class

The source class overloads a casting operator targeted to return an instance of the destination class.

```
class Product; // Forward declaration

// Source Class
class Inventory {
private:
    int items = 10;
public:
    // Overloaded conversion operator: Inventory -> Product
    operator Product() const;
};
```

Analogy & PU Question



Real-World Analogy

Operator overloading is like teaching a robot what '+' means for recipes. By default, + is not defined for Recipe objects. You define: 'Recipe + Recipe = combined ingredient list'. Now + means something meaningful for your type. The robot (compiler) knows exactly what to do.



Probable PU Exam Question

Q: What is operator overloading? Write a C++ program to overload '+' for complex numbers. Explain type conversion between objects and basic types. [PU Exam 2022]

Polymorphism

- Compile-time (Early) Binding vs Runtime (Late) Binding
- Virtual Functions — runtime dispatch
- Virtual Function Table (vtable) mechanism
- Pure Virtual Functions
- Abstract Classes


```
class Base {  
    virtual void control() = 0;  
};
```

```
class TV : public Base {  
    override void control();  
};
```

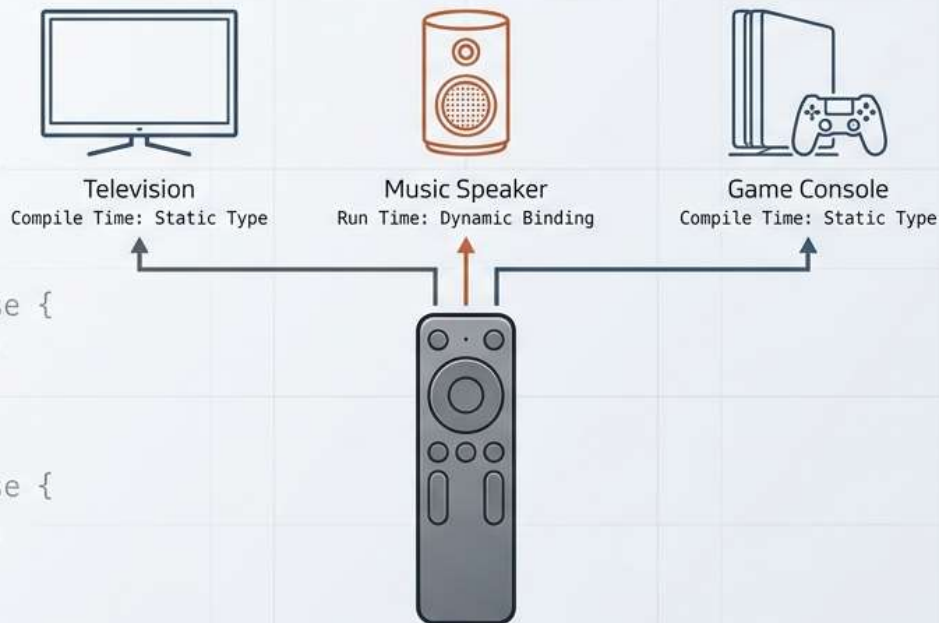
```
class TV : public Base {  
    override void control();  
};
```

```
class Speaker : public Base {  
    override void control();  
};
```

```
class Console : public Base {  
    override void control();  
};
```

The Architecture of Many Forms

A Visual Guide to C++ Polymorphism and Dynamic Binding



Polymorphism

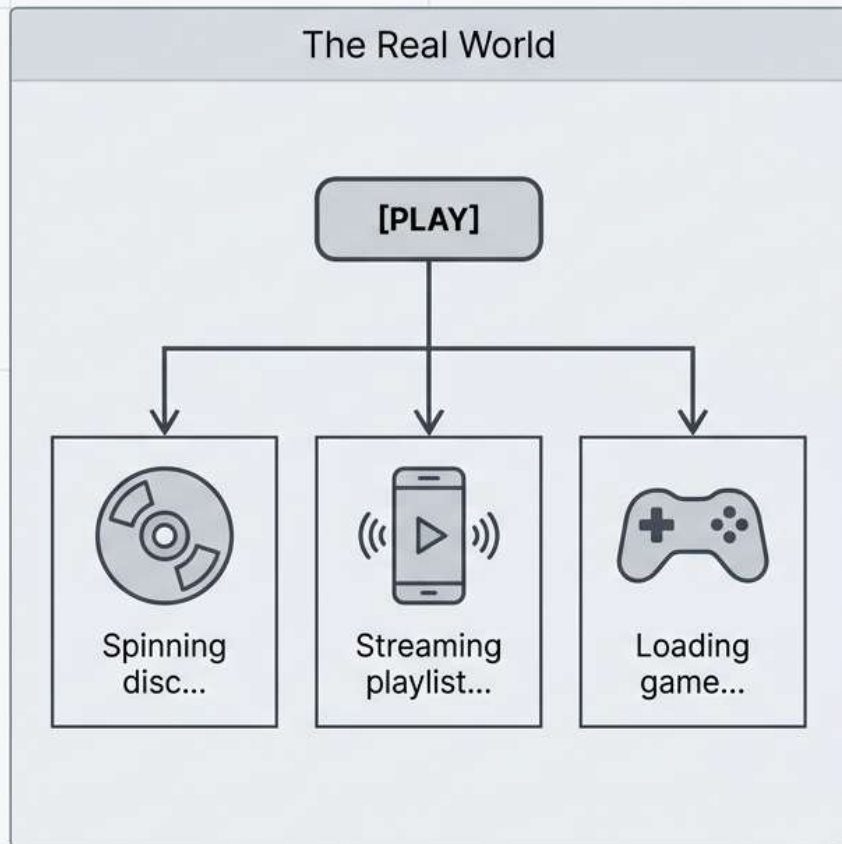
The remote-control analogy

Imagine you have a universal remote with a "**Play**" button. When you press it:

- **DVD Player** → Starts spinning the disc and playing a movie
- **Music Streaming App** → Begins streaming your playlist
- **Video Game Console** → Loads and starts the game

Same action (Play), different behaviors depending on what device you're pointing at. That's polymorphism!

One Interface, Multiple Behaviors



One Interface, Multiple Behaviors

The C++ Reality

```
Device* myDevice = getDevice();  
  
myDevice->play(); // What happens here?
```

The core philosophy of polymorphism is reducing cognitive load. You call `play()` on a generic pointer, and the system automatically routes the command to the correct behavior based on the specific device. The big question: **when does the system decide which path to take?**

Polymorphism

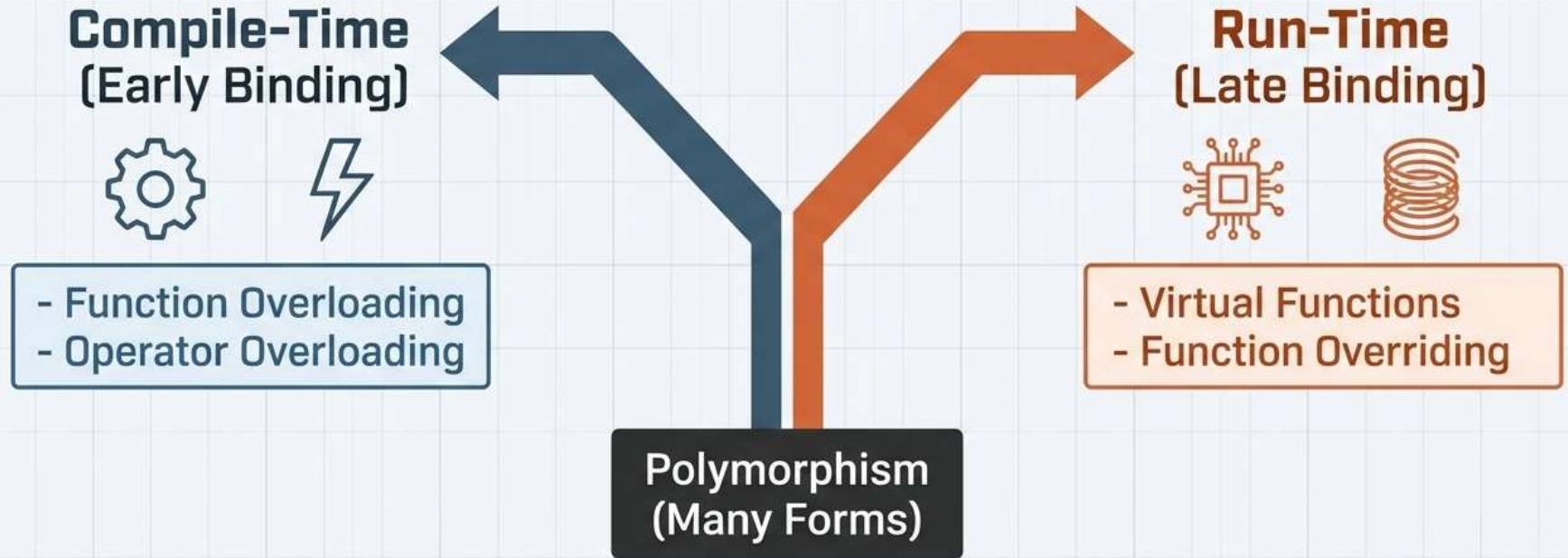
What is Polymorphism?

Polymorphism = "Poly" (many) + "Morph" (forms) = **Many forms**

In C++ OOP, polymorphism lets you treat different objects uniformly while each object performs its own specific version of an action.

Polymorphism stands as one of the four essential pillars of Object-Oriented Programming (OOP). To a Technical Architect, polymorphism is the strategic mechanism that allows for the creation of extensible and decoupled systems. It provides a single interface to a variety of underlying forms, ensuring that adding new features does not necessitate a complete rewrite of the core system logic.

The Grand Divide of C++ Execution



C++ empowers you to choose when function resolution happens: hardcoded before the program runs (Blue), or dynamically evaluated while running (Orange).

Classification of Polymorphism

Two completely different types of polymorphism

Compile-time Polymorphism (Static/Early Binding)

Compiler picks which function to call at compile time. Done through function overloading and operator overloading. Also called *early binding* or *static polymorphism*.

- function overloading
- operator overloading

Runtime Polymorphism (Dynamic/Late Binding)

Program decides which function to call while it is running. Done through virtual functions and base class pointers. Also called late binding or dynamic polymorphism.

- virtual functions
- base class pointer / Function

The key question that separates them

When does C++ decide which function to call — at compile time or at runtime?

If the answer is compile time → compile-time polymorphism. If runtime → runtime polymorphism.

Comparison Table: Binding Mechanisms

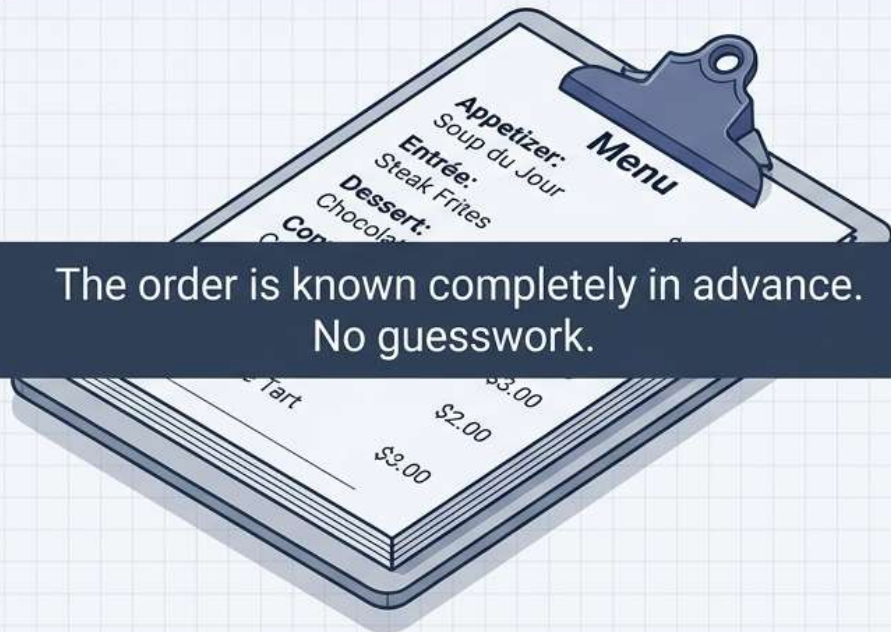
Feature	Compile-time (Static/Early Binding)	Run-time (Dynamic/Late Binding)
Resolution Timing	Occurs during the compilation phase.	Occurs during program execution.
Execution Speed	Faster; zero runtime overhead.	Slightly slower (vtable lookup overhead).
Flexibility	Fixed at compile time; less extensible.	Highly flexible; supports plugin architecture.
Implementation	Function/Operator Overloading.	Function Overriding/Virtual Functions.
Resolution Rule	Resolved via function signature.	Resolved via object type (vptr).

The "So What?" Layer

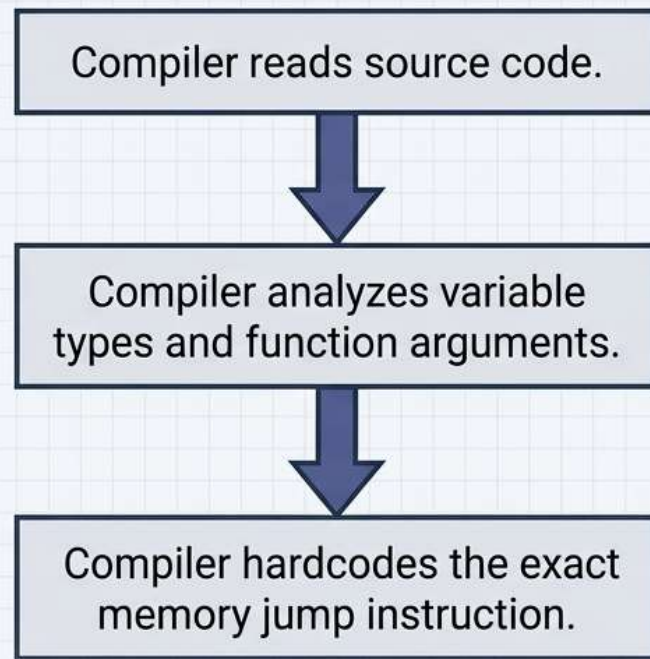
*The trade-off is efficiency versus adaptability. **Compile-time polymorphism** is preferred when performance is the absolute priority and the data types are known. **Run-time polymorphism** is essential for complex systems where the specific object type—such as a user-selected tool in a graphics suite—cannot be determined until the program is actually running.*

Early Binding Works Like a Fixed Menu

The Metaphor



The Mechanism



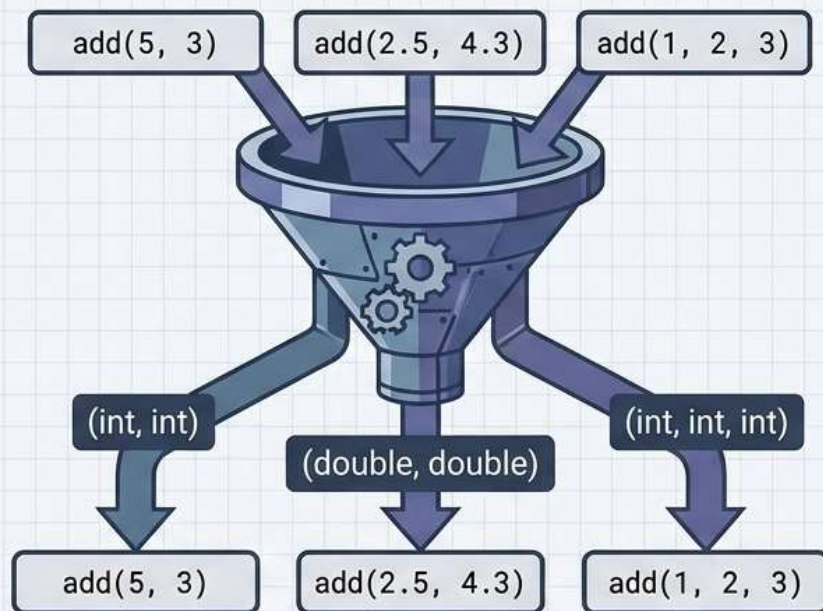
Zero Runtime Overhead. Blazingly Fast Execution. Decided before the program ever opens.

Function Overloading Sorts by Signature

The Code Definitions

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(double a, double b) {  
    return a + b;  
}  
  
int add(int a, int b, int c) {  
    return a + b + c;  
}
```

The Compile-Time Sorting Mechanism



The compiler treats these as entirely different functions based on their parameter lists. All routing is resolved purely at compile time.

Compile-time Polymorphism – Overloading (Early binding)

Overloading allows the same interface to handle different data types or parameter counts, streamlining the API for developers and ensuring intuitive syntax.

Function Overloading

Definition: Multiple functions share a name but differ in their signature (parameter type or count).

Architectural Warning:

Implicit Conversion A common pedagogical pitfall is ignoring **Implicit Conversion**.

If you define an `add(int, int)` function but pass double arguments (e.g., 2.5 and 4.3), C++ will implicitly convert them to integers, resulting in data loss (returning 6 instead of 6.8).

Overloading with a specific double version prevents this error.

```
class Calculator {  
public:  
    int add(int a, int b) { return a + b; }  
    // Overloading prevents implicit conversion/data  
    loss  
    double add(double a, double b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
};
```

Redefining Operators for Custom Types

obj1 + obj2

operator+

obj1.operator+(obj2)

```
class Complex {  
public:  
    // Redefining the '+' operator  
    Complex operator+(Complex obj) {  
        Complex res;  
        res.real = real + obj.real;  
        return res;  
    }  
};
```

SUMMARY

- **Concept:** Redefining native symbols (+, -, *, ==) for custom classes.
- **Purpose:** Makes custom objects behave intuitively, like primitive types.
- **Resolution:** The compiler translates the symbol and binds it to the custom function **at compile time**.

Compile-time Polymorphism – Overloading (Early binding)

Operator Overloading

Definition: Redefining the behavior of standard operators (e.g., +, -) for user-defined classes using the operator keyword.

Architectural Warning:

Implicit Conversion A common pedagogical pitfall is ignoring **Implicit Conversion**.

If you define an `add(int, int)` function but pass double arguments (e.g., 2.5 and 4.3), C++ will implicitly convert them to integers, resulting in data loss (returning 6 instead of 6.8).

Overloading with a specific double version prevents this error.

The "So What?" Layer

Overloading enhances code readability by allowing intuitive syntax like `s1 + s2` rather than `s1.add(s2)`. This aligns software logic with mathematical expectations, making the codebase more maintainable for large teams.

```
#include <iostream>
using namespace std;

class Score {
    int val;
public:
    Score(int v) : val(v) {}
    // Operator Overloading Syntax: Return_Type
    operator Symbol()
    void operator-() {
        val = -val;
    }
    void display() { cout << "Value: " << val << endl;
}
};

int main() {
    Score s1(50);
    -s1; // Calls operator-()
    s1.display(); // Output: Value: -50
    return 0;
}
```

Late Binding is the Chef's Special



The exact dish isn't known until you order it based on fresh ingredients.

Program is actively running.

Code requests a function via a generic pointer.

System pauses to inspect the ACTUAL object in memory.

System dynamically routes to the correct function.

Requires immense flexibility. The decision of which function to call is deferred to the exact moment of execution.

Run-time Polymorphism – Overriding (Late binding)

What is Overriding?

Definition: A derived class provides its own implementation of a virtual function from the base class.

Rules of Overriding:

Function must be virtual in base class

Same name, return type, and parameters

Run-time Polymorphism – Overriding (Late binding)

Run-time polymorphism enables dynamic behavior in class hierarchies, allowing a child class to provide a specific implementation of a parent's function.

Function Overriding Explained

Definition: A derived class provides a new definition for a base class function. To succeed, the **Same-Signature Rule** must be followed: name, parameters, and return types must match exactly.

Step-by-Step Logic:

Inheritance: A Child class inherits from a Parent class.

Definition: Both classes define void display().

Execution: When a Child object calls display(), the local version takes precedence.

Mechanism: The child's version "discards" the parent's version during execution as the compiler resolves the call based on the local scope of the object.

```
#include <iostream>
using namespace std;

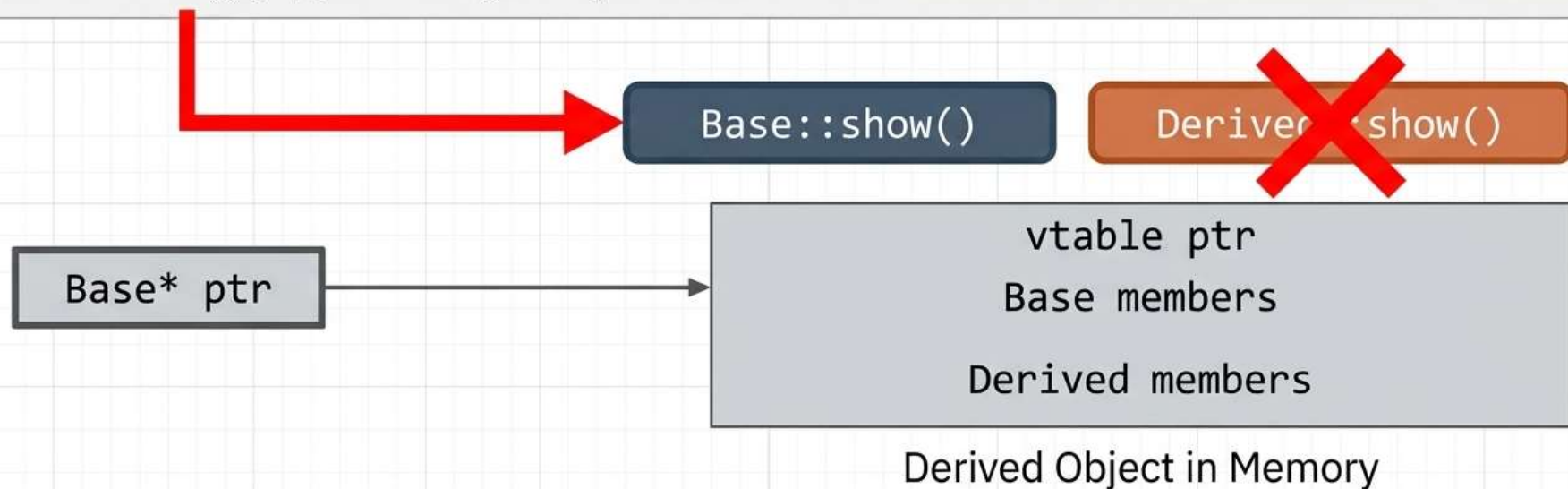
class A {
public:
void display() { cout << "I am Class A " << endl; }
};

Class B : public A {
public:
void display() { cout << "I am Class B" << endl; }
};

int main() {
    B b1;
    b1.display();
    return 0;
}
```

The Pointer Problem: When the Compiler Assumes Too Much

```
Base* ptr = new Derived();  
ptr->show(); // What gets printed?
```



Without intervention, C++ defaults to **Early Binding**. It looks at the **POINTER POINTER TYPE** (Base), not the **ACTUAL OBJECT TYPE** (Derived) it points to.

Run-time Polymorphism – Overriding (Late binding)

```
#include <iostream>
using namespace std;

class Base {
public:
void display() { cout << "I am Base Class" << endl; }
};

Class Derived : public Base {
public:
void display() { cout << "I am Derived Class" << endl; }
};

int main() {
    Base* ptr;
    Derived d;
    ptr = &d

    bptr->d;
    return 0;
}
```

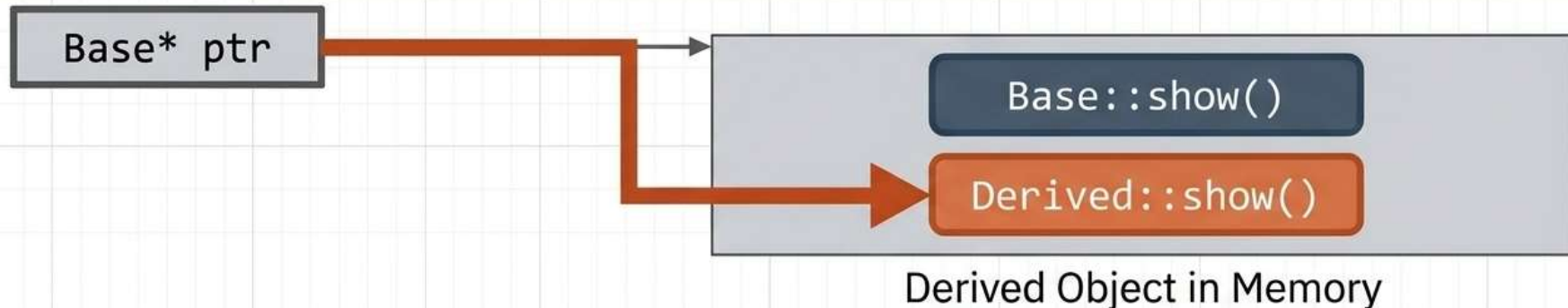
Forcing Late Binding with Virtual Functions

Base Class

```
class Base {  
public:  
    virtual  
    void show() { // Wait until runtime!  
        cout << "Base class";  
    }  
};
```

Derived Class

```
class Derived : public Base {  
public:  
    void show()  
    override { // Replaces base behavior  
        cout << "Derived class";  
    }  
};
```



> Output: Derived class

Run-time Polymorphism – Overriding (Late binding)

A **virtual function** is a member function which is declared within base class and re-defined(Overridden) by derived class

When we refer to a derived class object using pointer or a reference to the base class, we can call a virtual function for that object and execute the derived class's version of the function.

- Virtual functions ensures that the correct function is called for an object, regardless of the type of reference(or pointer) used for function call.
- They are mainly used to achieve Runtime polymorphism
- Functions are declared with virtual keyword in base class.
- The resolving of function call is done at Run-time.

Virtual Functions

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
    // Virtual function enables overriding in
    derived classes
    virtual void speak() {
        cout << "Animal makes a sound" << endl;
    }

    // Pro Tip: Always use a virtual destructor in
    base classes to prevent memory leaks virtual
    ~Animal() {}
};

// Derived class
class Dog : public Animal {
public:
    // 'override' keyword (C++11+) ensures the
    signatures match correctly
    void speak() override {
        cout << "Woof! Woof!" << endl;
    }
};
```

```
int main() {
    // 1. Create a base class pointer
    Animal* ptr;

    // 2. Create an object of the derived class
    Dog myDog;

    // 3. Point the base pointer to the derived class object
    ptr = &myDog;

    /* 4. Call the virtual function through the pointer
    Because 'speak' is virtual, the program performs "late
    binding" and calls the Dog version, not the Animal
    version.*/

    ptr->speak();

    return 0;
}
```

Virtual Functions & Abstract Classes

- Virtual keyword in base class enables runtime dispatch
- Base class POINTER can call derived class methods
- override keyword (C++11) catches spelling mistakes
- Pure virtual: virtual void func() = 0;
- Makes class ABSTRACT — cannot be instantiated
- All pure virtual functions MUST be implemented in derived
- vtable: Compiler creates table of function pointers per class
- Each object has a vptr pointing to its class vtable
- Runtime lookup: ptr → vptr → vtable → correct function

```
class Shape {
public:
    virtual void draw() = 0; // Pure virtual
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Circle" << endl;
    }
};

int main() {
    Shape *p = new Circle;
    p->draw(); // Runtime: Circle
}
```

Analogy & PU Question



Real-World Analogy

Virtual functions are like a 'Speak' command to different animals. You tell any animal to 'Speak' (same command through a base pointer). A Dog barks, a Cat meows, a Duck quacks. The actual sound is determined at RUNTIME based on the real animal — not at compile time based on what you called it.



Probable PU Exam Question

Q: What is runtime polymorphism? How is it achieved using virtual functions? Explain pure virtual functions and abstract classes. [PU Exam 2023]

Inheritance

- Base and Derived Classes
- Single, Multiple, Multilevel, Hierarchical, Hybrid
- Private, Protected, Public Inheritance
- Function Overriding
- Virtual Base Class — solving the Diamond Problem
- Constructor/Destructor in Derived Classes

5 Forms of Inheritance

- Single: One derived $\leftarrow$ One base (Dog : Animal)
- Multiple: One derived $\leftarrow$ Many bases (Duck : Flying, Swimming)
- Multilevel: Chain — C : B : A (GrandChild : Child : Parent)
- Hierarchical: Many derived $\leftarrow$ One base (Cat, Dog, Bird : Animal)
- Hybrid: Combination of two or more forms
- Diamond Problem:
 - When B and C both inherit A, and D inherits B and C $\rightarrow$ two copies of A
 - Solution: virtual public inheritance in B and C

```
// Single
class Dog : public Animal {};

// Multiple
class Duck: public Flying,
           public Swimming {};

// Virtual base (Diamond fix)
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {
    // Only ONE copy of A
};
```

Ch.8 Analogy & PU Question



Real-World Analogy



Inheritance is like a family tree. A child inherits traits from parents — eye color, height. But the child can also have unique traits or override inherited ones (choosing a different hairstyle). The diamond problem is like when two parents share the same grandparent — you'd only want ONE set of grandparent traits in the grandchild, not two duplicates.



Probable PU Exam Question

Q: What is the diamond problem in C++ inheritance? How is it solved using virtual base classes? Explain all 5 forms of inheritance with diagrams. [PU Exam 2021]

File Handling

- Stream Class Hierarchy: ios → fstream
- Opening files: constructor and open() method
- File Modes: in, out, app, trunc, binary, ate
- File Pointers: seekg, seekp, tellg, tellp
- Error Handling: good(), bad(), fail(), eof()

File Handling — Complete Overview

- ifstream — file input (reading)
- ofstream — file output (writing)
- fstream — both reading and writing
- File Modes:
 - ios::in = read | ios::out = write | ios::app = append
 - ios::trunc = overwrite | ios::binary = binary mode
- File Pointers:
 - seekg(n, ios::beg/cur/end) — move read pointer
 - seekp(n, ios::beg/cur/end) — move write pointer
 - tellg() / tellp() — get current position

```
#include <fstream>
using namespace std;

int main() {
    ofstream out("data.txt");
    out << "Hello File!" << endl;
    out.close();

    ifstream in("data.txt");
    if (!in) { cout << "Error!"; }
    if (in.good()) cout << "OK";
    if (in.eof()) cout << "EOF";
    in.close();
}
```


Ch.10 Analogy & PU Question



Real-World Analogy

File pointers are like bookmarks in a physical book. `seekg()` moves your reading bookmark, `seekp()` moves your writing bookmark. `tellg()` tells you which page you're on. `ios::app` is like opening a diary and writing at the last used page — you never overwrite old entries.



Probable PU Exam Question

Q: Write a C++ program to read student records from a file, search by roll number, and display the result. Explain file pointer manipulation functions with examples. [PU Exam 2022]

Advanced C++ Topics

- Function Templates and Class Templates
- Standard Template Library (STL) Introduction
- Namespaces — using directive vs using declaration
- Exception Handling: try, throw, catch
- Creating and Using Header Files

Templates & Namespaces

- Function Template: Same algorithm, any data type
- Class Template: Generic container/data structure
- STL uses templates: `vector<T>`, `stack<T>`, `map<K,V>`
- Namespace: Prevents name collisions between libraries
- `using namespace X;` — import all names from X
- `using X::name;` — import only one specific name
- Nested namespaces allowed in C++17 (`A::B::C`)

```
// Function template
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

// Class template
template <typename T>
class Stack {
    T data[100]; int top=-1;
public:
    void push(T v) { data[++top]=v; }
    T pop() { return data[top--]; }
};
```

Exception Handling: try-throw-catch

```
#include <iostream>
#include <stdexcept>
using namespace std;

float divide(float a, float b) {
    if (b == 0)
        throw runtime_error("Div by zero!");
    return a / b;
}

int main() {
    try {
        cout << divide(10, 2);    // OK
        cout << divide(5, 0);    // Throws
    }
    catch (const runtime_error &e) {
        cout << "Error: " << e.what();
    }
    catch (...) { // Catch-all
        cout << "Unknown error";
    }
}
```

💡 Exceptions separate error handling from normal logic. Never use exceptions for normal flow control.

Ch.11 Analogy & PU Question



Real-World Analogy

Templates are like cookie cutters: one cutter works for any type of dough (data type). Namespaces are like area codes in phone numbers — (01) Kathmandu::Ram is different from (061) Pokhara::Ram. Exception handling is like a safety net in a circus: performers (code) do their act, and if they fall (error), the net (catch) prevents catastrophe.



Probable PU Exam Question

Q: What are templates in C++? Differentiate function templates and class templates. Write a C++ program demonstrating exception handling with multiple catch blocks. [PU Exam 2023]

Thank You!

OOP with C++ — Complete Course Module

Purwanchal University, Nepal | Chapters 1–11 | 41 Credit Hours